

AI生图范式演进与里程碑论文解读 | 从VAE、GAN到 Diffusion、自回归

简介

AI 图像生成技术在过去十余年间经历了多次范式转换。从早期的 **VAE 和 GAN** 开创了“让神经网络学会生图”这条道路；再到 2020 年之后**扩散模型**在质量和稳定性上的飞跃；再到**自回归模型**试图用大语言模型的统一框架吞并图像生成，每一步都是在前一步的瓶颈之上寻找全新的突破口。本文系统梳理了该领域的 **17 篇里程碑论文**，力求呈现一条清晰的技术演进脉络。

一、早期范式

生成式 AI 的故事要从两篇开创性论文开始：**VAE 和 GAN** 分别提出了两种**截然不同的**图像生成思路，它们开创的核心概念「**潜空间**」和「**对抗训练**」至今仍是整个领域的基石。



假设你有一万张人像图片，你想训练一个 AI，让它能凭空画出一个从未存在过的人。

1. VAE (2013)

VAE — Variational Autoencoder (2013)

Auto-Encoding Variational Bayes — Kingma & Welling (2013)<https://arxiv.org/abs/1312.6114>

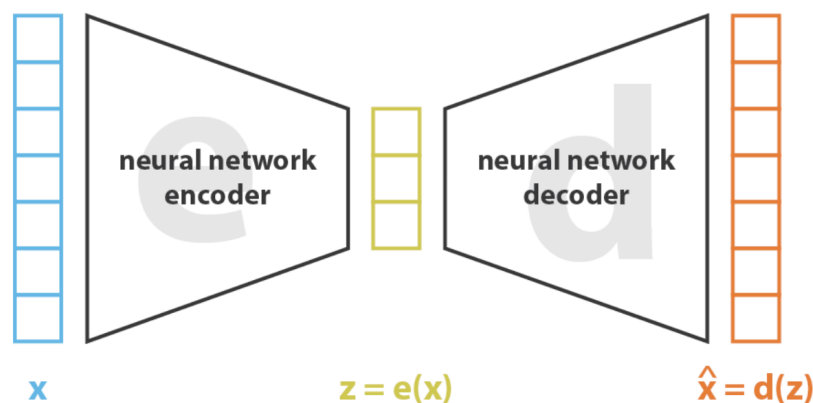


经典生图范式；引入「潜空间」框架思想。

从 Autoencoder 开始

Autoencoder 是一种无监督神经网络，结构分为**编码器**和**解码器**两部分。编码器将输入压缩为低维表示（潜在向量），解码器再从该表示重建原始输入。训练目标是让输出尽可能接近输入。

核心用途包括**降维**、去噪、特征学习和生成模型（如 VAE）。可以把它理解成为一种“学会压缩”的网络，迫使模型在瓶颈层捕捉数据最本质的特征。



$$\text{loss} = \|x - \hat{x}\|^2 = \|x - d(z)\|^2 = \|x - d(e(x))\|^2$$

autoencoder 和它的损失函数

显变量与隐变量

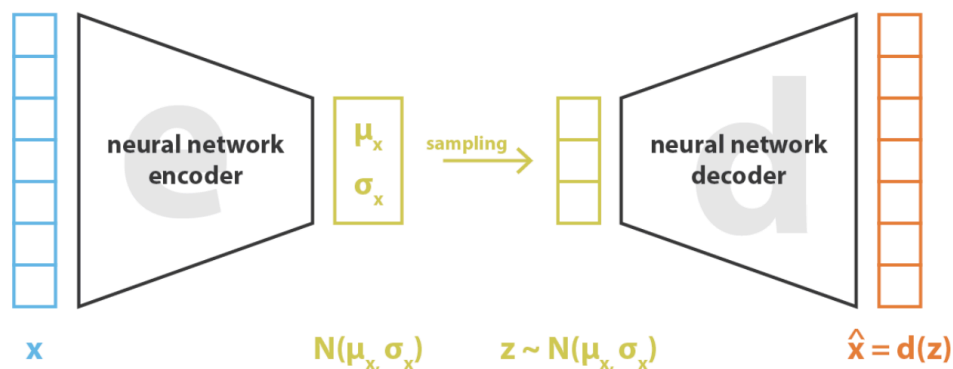
- **显变量(observed variable,记作 x)**: 能直接被观测到的原始数据本身。一张图片的几十万个像素值、一段声音的波形采样, 等等。显变量就是 Autoencoder 的输入和输出。
- **隐变量(latent variable,记作 z)**: 藏在数据背后但不直接出现在数据里的核心因素。以手写数字图片为例, 显变量是 28×28 个像素, 而真正决定这张图的只有少数几个隐藏因素: 这是数字几、笔画多粗、字体倾斜程度, 等等。
- **两者的关系可以理解成一种"压缩与还原"**。一张图片表面上是几十万个杂乱的数字(显变量), 但它承载的有效信息其实很少, 可以被概括成寥寥几个有意义的因素(隐变量)。Autoencoder 本质就是在这两层之间架桥:
 - **编码器(encoder)= 从显变量提炼隐变量**。输入一堆像素 x , 输出一个低维的潜在向量 z ——相当于把"看得见的表面"压缩成"看不见的本质"。
 - **解码器(decoder)= 从隐变量还原显变量**。输入潜在向量 z , 重建出像素 x ——相当于把"本质"再展开回"表面"。

Autoencode 训练过程

1. **前向传播**: 从训练集采样一批图片 → 编码器输出潜在表示 → 解码器输出重建图像。初始阶段参数随机, 编码无意义, 重建接近噪声。
2. **计算损失**: 逐像素比对重建图与原图, 算出差距 (Loss) 。
3. **反向传播**: 通过 BP 算法计算梯度, 将编码器与解码器视为一个整体网络, 一次性更新所有参数。
4. **重复迭代**: 以上过程重复 N 轮, 损失持续下降, 重建质量逐步提升。

VAE (Variational Autoencoder, 变分自编码器) 简介

VAE 的核心思想是: 通过 **encoder** 将图像压缩到一个**连续的潜空间** (latent space), 再用 **decoder** 从潜空间采样重建数据。它确立了「编码器 → 潜空间 → 解码器」这一架构范式。

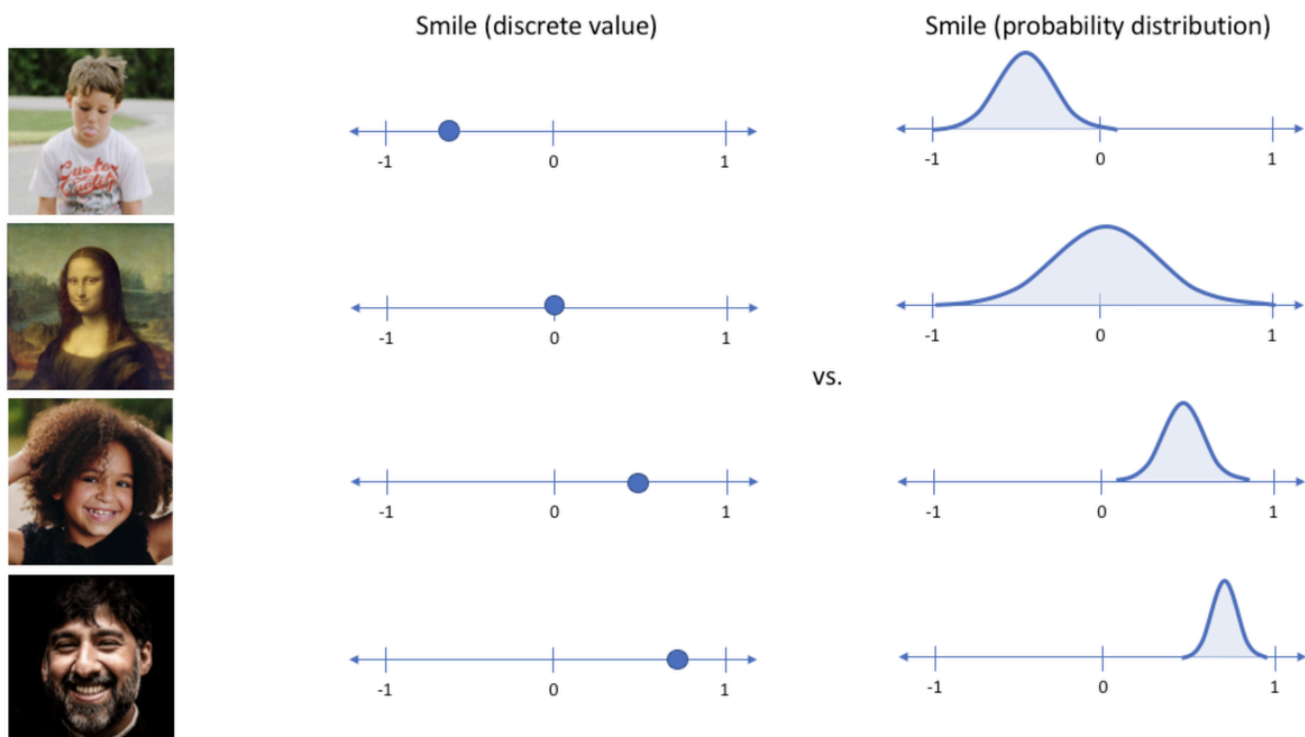


$$\text{loss} = \|x - \hat{x}\|^2 + \text{KL}[N(\mu_x, \sigma_x), N(0, I)] = \|x - d(z)\|^2 + \text{KL}[N(\mu_x, \sigma_x), N(0, I)]$$

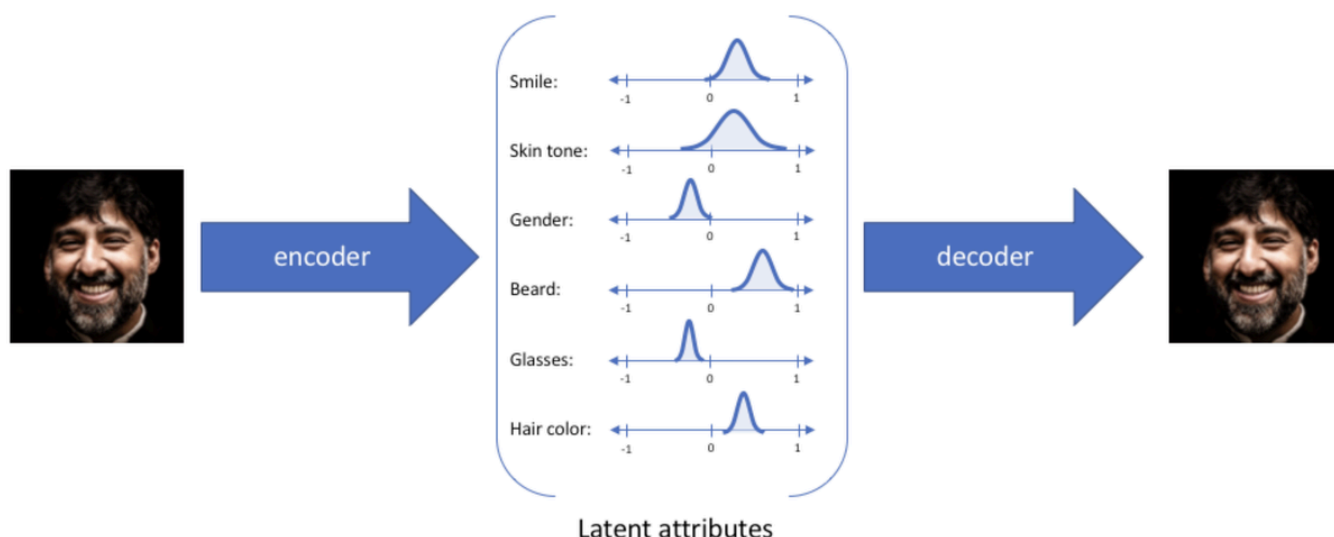
VAE 和它的损失函数

VAE 与 Autoencoder 的关键区别

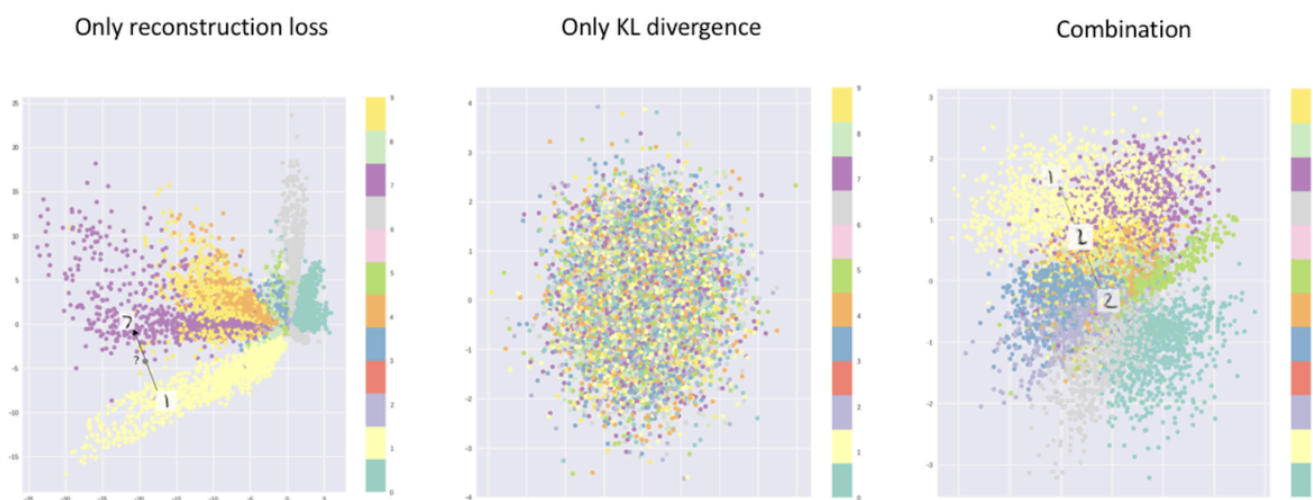
- **输出方式不同：** Autoencoder 的编码器输出一个**固定的潜在向量**（一组确定的数字），VAE 的编码器输出的是一个**概率分布的参数**（均值 μ 和方差 σ ），再从这个分布中采样得到潜在向量。
- **训练目标不同：** Autoencoder 只有重建损失（输出尽量接近输入）。VAE 在此基础上增加了 **KL 散度损失**，强制让编码器输出的分布接近标准正态分布，从而使潜在空间连续、平滑。



同样是「笑的程度」，普通 Autoencoder（中间）存一个固定值，VAE（右边）存一条概率分布——曲线的中心 (μ) 表示笑的程度（越靠右越笑），胖瘦 (σ) 表示确定程度（越窄越确定、越宽越模糊）。比如大笑的男人中心靠右且曲线窄，蒙娜丽莎中心居中且曲线宽（似笑非笑、模棱两可）。这就是两者的根本区别：VAE 编码器输出的是分布参数，而非确定的数字。



VAE 的完整流程：人脸经 encoder 压进潜空间，再由 decoder 还原。潜空间里存的不是数字，而是一组分布（笑、肤色、性别、发色……）。正因存的是分布，模型才能采样、才能生成；也正因分布连续，潜空间才平滑。注意：真实维度并不会这么整齐地对应"笑""胡子"这类属性，真实的特征是高维空间中的抽象数学表达。此处仅为示意。

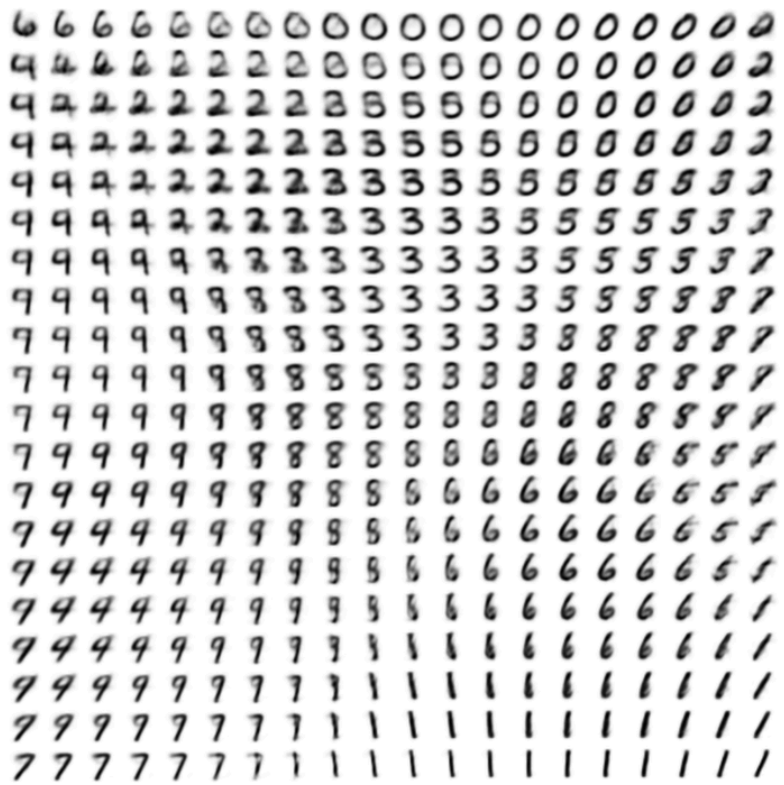


VAE 核心贡献

- **确立「编码器 → 潜空间 → 解码器」范式：**在 VAE 之前，Autoencoder 只能压缩还原，不能生成。VAE 把变分推断引入这套架构，第一次让"编码-解码"这条路线具备了生成能力。
- **让潜在空间可采样。**从压缩工具变为生成模型。编码器输出分布而非固定值，加上 KL 散度约束，使潜在空间连续、平滑，任意采样都能生成合理的输出。
- **重参数化技巧 (Reparameterization Trick)。**"从分布中采样"这个操作本身不可导，无法反向传播。VAE 提出将采样过程改写为 $z = \mu + \sigma \cdot \epsilon$ (ϵ 从标准正态采样)，把随机性剥离到外部，使梯度能顺畅地流过整个网络。这个技巧后来被广泛复用。
- **将神经网络与贝叶斯概率框架结合。**首次优雅地把深度学习的表达能力和概率生成模型的数学严谨性统一在一个端到端可训练的框架里。



(a) Learned Frey Face manifold



(b) Learned MNIST manifold

VAE 用 2 维潜空间画的图。在潜空间里均匀取一堆点,每个点都用 decoder 解码成一张图,再按位置摆好。可以看到相邻的图是缓慢渐变的:人脸的朝向、表情连续变化,数字也是一个慢慢变成另一个,中间没有断裂。这说明 VAE 的潜空间是连续平滑的,随便取一点都能生成合理的图,这正是它能随机生成、能平滑过渡的原因。

VAE 局限

- **生成图像模糊:** 由于使用像素级重建损失 (MSE) 以及 KL 散度对隐空间的过度正则化, VAE 生成的图像往往偏模糊, 缺乏高频细节。

VAE 地位 & 影响

VAE 是整个生成模型领域的基石之一。

它引入的「潜空间」概念被后来的 **Stable Diffusion** (Latent Diffusion Model) 直接继承;

其变体 **VQ-VAE** 将连续潜变量离散化成「视觉词典」, 为**自回归**图像生成铺平了道路。

可以说, VAE 虽然在生图质量上很快被 GAN 超越, 但其**"编码器-隐空间-解码器"**的框架思想贯穿了此后几乎所有主流生成架构。

VAE 参考文档

<https://bluefisher.github.io/2020/02/07/%E7%90%86%E8%A7%A3-Variational-Autoencoders-VAEs/> (推荐)

<https://www.jeremyjordan.me/variational-autoencoders/>

<https://www.ibm.com/cn-zh/think/topics/variational-autoencoder#186915249>

<https://www.vectorexplorer.com/tech/auto-encoder/vae.html>

<https://zhouyifan.net/2022/12/19/20221016-VAE/>

2. GAN (2014)

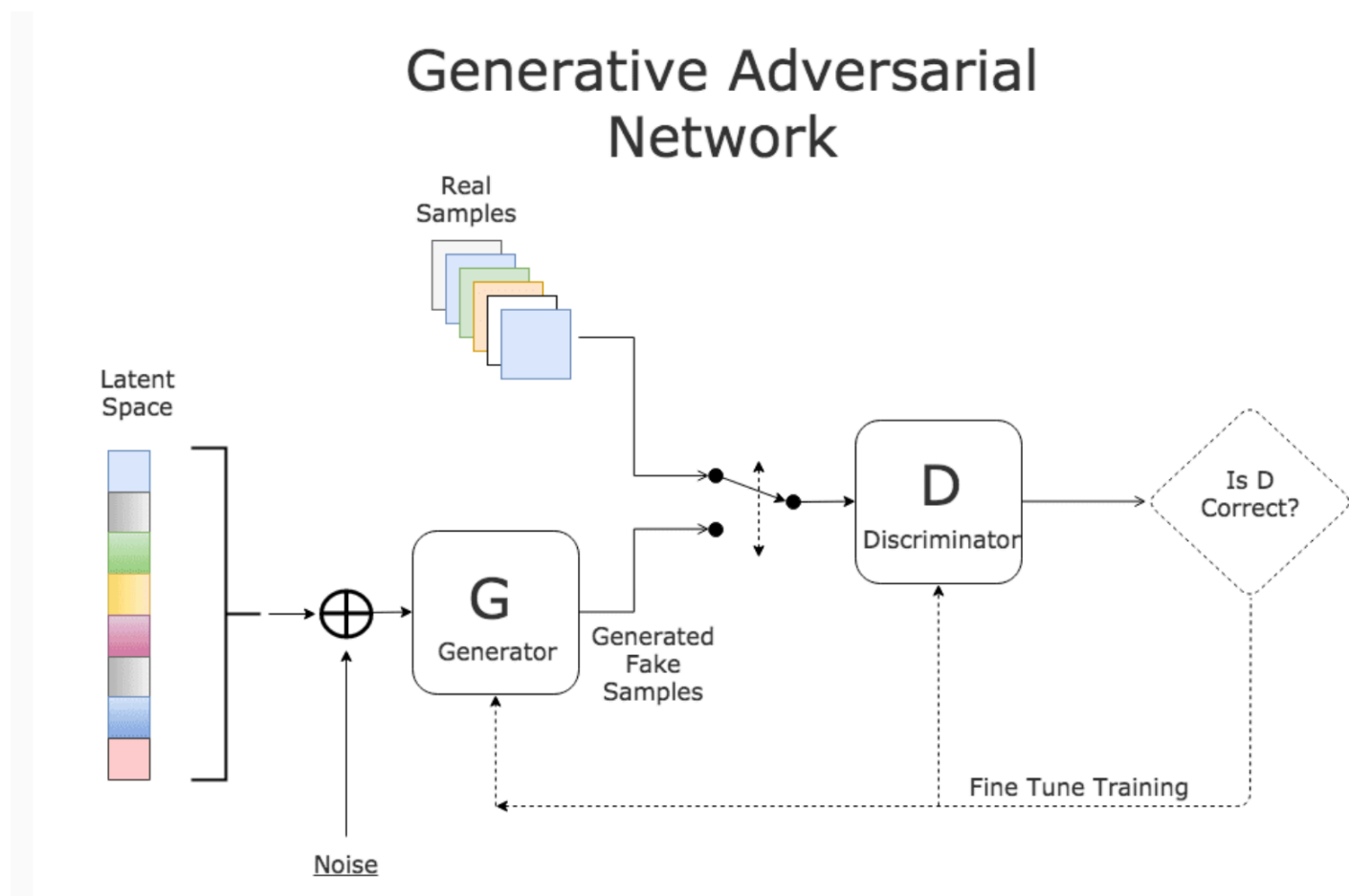
GAN — Generative Adversarial Network (2014)

Generative Adversarial Nets — Goodfellow et al. (2014) <https://arxiv.org/abs/1406.2661>

💡 经典生图范式；训练「生成器」和「判别器」。

简介

生成对抗网络（Generative Adversarial Network, GAN）提出了一个全新的生成范式：通过两个网络的博弈来学习数据分布。**生成器（Generator）**试图从随机噪声生成逼真的假样本，**判别器（Discriminator）**试图区分真假样本。两者交替优化，最终达到纳什均衡——生成器生成的样本足以“骗过”判别器。



GAN 网络架构

GAN 网络结构

- **生成器：**接收一个固定维度的随机噪声向量（如 100 维）作为输入，经过若干层网络逐步上采样，输出一张与训练数据同尺寸的图像。原始论文使用全连接层（MLP），DCGAN（2015）将其替换为转置卷积网络，成为后续 GAN 变体的标准架构。

- **判别器：** 接收一张图像作为输入，经过若干层网络逐步下采样，最终输出一个 0-1 的标量概率值，表示该图像为真实样本的置信度。结构与生成器大致对称，原始论文使用全连接层（MLP），DCGAN 后改为卷积网络。

GAN 的训练过程

- **损失函数：** 判别器和生成器共享同一个 min-max 目标函数。判别器想最大化这个目标（正确区分真假），生成器想最小化它（让判别器犯错）。
- **前向传播：**
 - 生成器生成：每轮将随机噪声送入生成器得到假样本。
 - 判别器判别：再将假样本与真样本一起送入判别器，判别器对每张图输出真假概率。
- **反向传播：**
 - 判别器更新：固定生成器，根据判别器损失对判别器做反向传播，更新其参数，使其判断更准确。
 - 生成器更新：固定判别器，将梯度从判别器的打分一路回传至生成器，更新生成器参数，使其生成的假样本更逼真。

核心贡献

- **对抗训练框架：** 无需显式地定义数据的似然函数，也无需变分近似，通过 min-max 博弈隐式地学习数据分布。这是生成模型思路上的根本性创新。
- **生成质量飞跃：** 与同期的 VAE 相比，GAN 生成的图像更加锐利、逼真，开启了深度生成模型在视觉上"可用"的时代。
- **框架通用性：** GAN 的对抗思想可以与几乎任何网络架构和损失函数结合，催生了大量变体（DCGAN、WGAN、StyleGAN、BigGAN 等）。

局限

- **训练不稳定：** 生成器和判别器的平衡极难维持，容易出现梯度消失、模式坍缩（Mode Collapse，生成器只产出少量模式）等问题。
- **多样性不足：** 模式坍缩意味着 GAN 难以覆盖数据分布的所有模式，生成样本的多样性常低于 VAE 和扩散模型。

地位 & 影响



StyleGAN 的风格混合效果。顶部一行(Source B)与最左一列(Source A)是各自独立生成的人脸,其余每张图都是把 Source B 的"粗粒度风格"(姿态、脸型、发型等)迁移到 Source A 上生成的结果。可以看到生成的人脸高度逼真、细节自然(肤色、发丝、光照、表情皆栩栩如生),且能在不同人脸间自由迁移与组合特征——这些人物均不真实存在,全部由 GAN 凭空生成,充分展现了 GAN 强大的图像生成能力。

图片来源:Karras et al., A Style-Based Generator Architecture for Generative Adversarial Networks (StyleGAN), CVPR 2019, NVIDIA. arXiv:1812.04948

GAN 在 **2014–2020** 年间是图像生成领域的**绝对主流范式**。从 DCGAN (2015) 引入卷积架构,到 StyleGAN (2018/2019) 实现人脸生成的照片级逼真度,再到 BigGAN (2018) 在 ImageNet 上生成高分辨率图像,GAN 家族推动了生成质量的持续突破。

然而,GAN 的训练不稳定和模式坍塌问题始终未被根本解决,这为后来扩散模型的崛起埋下了伏笔。

GAN 的对抗训练思想也被后续大量工作借用。最典型的例子是 **VQGAN** (2021)。它把 GAN 的对抗损失引入 **VQ-VAE 的 tokenizer 训练**中,大幅提升了图像离散化的质量,成为**自回归范式**的关键组件。

2021 年,Diffusion Models Beat GANs 一文的诞生标志着 GAN 在无条件/有条件图像生成上的统治地位正式被**扩散模型**终结。

GAN 参考文档

<https://lilianweng.github.io/posts/2017-08-20-gan/>

<https://jonathan-hui.medium.com/gan-whats-generative-adversarial-networks-and-its-application-f39ed278ef09>

<https://towardsai.net/p/machine-learning/diffusion-models-vs-gans-vs-vaes-comparison-of-deep-generative-models>

二、基础工作

这两篇论文不直接属于任何生成范式，但它的研究成果对后续扩散和自回归两个范式的发展都起到了极为关键的作用。

1. ViT (2020)

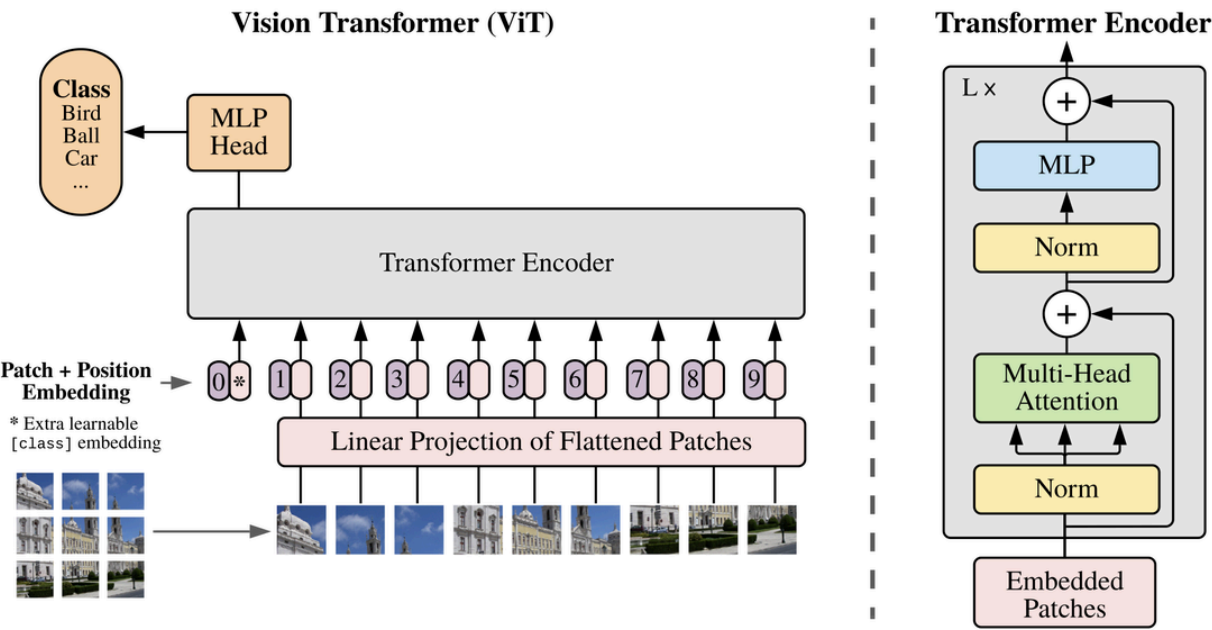
ViT — Vision Transformer (2020)
An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale — Dosovitskiy et al. (2020)
<https://arxiv.org/abs/2010.11929>

💡 证明了 Transformer 在视觉任务上的可行性。

简介

ViT 把图像识别（分类）这个任务，第一次完全用标准 **Transformer** 来做，不掺任何卷积，并且证明只要数据够多，它能超过当时最好的 CNN。是将 NLP 领域大获成功的 Transformer 首次"纯粹地"应用于计算机视觉的标志性工作

ViT 的做法极为简洁：将图像切分为固定大小的 patch（例如 16×16 ），每个 patch 线性投影成一个 token，然后用标准 Transformer 处理这些 token 序列。在足够大的数据上训练后，ViT **超越了所有 CNN 模型**。



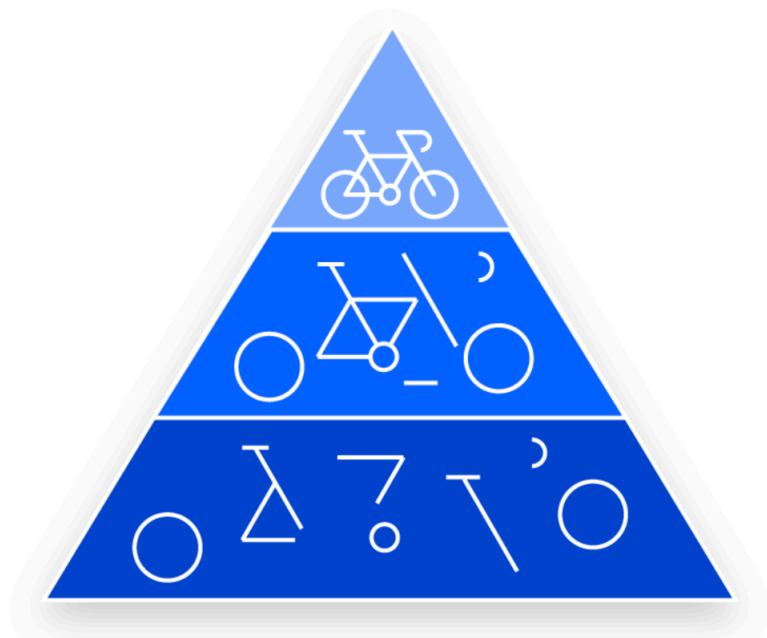
ViT 模型概览：将一张图像切分为固定大小的 patch，对每个 patch 做线性嵌入，加上位置编码，然后将得到的向量序列送入标准的 Transformer 编码器。

核心贡献

- **证明了纯 Transformer 在视觉任务上的可行性：** 在足够大的数据集上预训练后，ViT 在 ImageNet 上超越了最好的 CNN 模型，打破了"视觉任务必须用卷积"的共识。
 - 补充：什么是卷积神经网络（CNN）？

代码块

- 1 CNN 是在 ViT 之前统治计算机视觉十多年的主流架构。
- 2
- 3 **核心机制：** CNN 的基本操作是用一组小的"卷积核"（通常为 3×3 大小）在图像上逐步滑动扫描，每次只关注局部的一小块区域，提取该区域内的特征。不同的卷积核会关注不同类型的特征——边缘、纹理、颜色变化等。每一层通常同时使用几十到几百个卷积核，从不同角度提取信息。
- 4
- 5 **逐层抽象：** CNN 的核心逻辑是"局部扫描 → 逐层组合 → 从细节到整体"。浅层网络识别线条和边缘，中间层把线条组合成形状和纹理，深层进一步组合成物体的局部结构。每一层的特征都比上一层更抽象、视野也更大，最终从原始像素中逐步识别出完整的物体。
- 6
- 7 **局限：** CNN 的每一层只看局部区域，必须通过堆叠很多层才能间接获得全局视野。捕捉图像中远距离的关联（如图片左上角的物体与右下角的物体之间的关系）需要经过层层传递，效率较低。而 Transformer 通过注意力机制，让每个 token 在第一层就能直接与所有其他 token 建立关联，全局关系一步到位。这也是为什么在数据足够充分的条件下，ViT 能够超越所有精心设计的 CNN 模型。



这张图用一辆自行车说明 CNN 的核心机制——特征的逐层抽象。CNN 由多层堆叠而成，图像逐层向前传播，每经过一层，提取到的特征就更抽象、感受野也更大。

从下往上对应网络从浅层到深层：浅层（底部）只能提取最低级的局部特征——边缘、短线、圆弧，单看完全看不出是什么；中间层把这些低级特征组合成车轮、车架等更高级的局部结构；深层（顶部）进一步组合，最终对应到"自行车"这个完整物体。

这种"局部特征 → 逐层组合 → 整体语义"正是 CNN 的核心逻辑，也暴露了它的短板：每层的卷积核只覆盖局部感受野，全局信息必须靠堆叠足够多的层、层层传递才能获得。这也是为什么后来 Transformer（每个 token 第一层就能与全图交互）在大数据下能反超 CNN。

- **Patch Tokenization:** 将图像视为"patch 序列"的思想，为后来的 DiT（用 Transformer 替代 U-Net 做扩散）和自回归图像生成（把图像当 token 序列逐个预测）奠定了基础。
- **扩展性 (Scalability) :** 展示了 Transformer 在视觉领域同样遵循"更大模型 + 更多数据 = 更好性能"的 Scaling Law。

局限

- **数据饥渴:** 在中等规模数据集（如仅用 ImageNet-1k 训练）上，ViT 的表现不如同规模的 CNN，需要大规模数据或强数据增强才能发挥优势。
- **计算成本高:** 自注意力的二次复杂度使得处理高分辨率图像的计算成本很高。

地位 & 影响

ViT 的意义远超图像理解本身——它证明了 **Transformer** 可以作为视觉领域的通用骨干，从而打开了**视觉与语言共享统一架构**的大门。

这直接促成了 **CLIP**（视觉-语言对齐）、**DiT**（用 Transformer 替代 U-Net 做扩散模型骨干）、以及各种自回归视觉生成模型（**DALL·E 1**、**LlamaGen** 等）。

可以说，「图像可以被当作一串 token 来处理」这个思想，是后来**从扩散模型走向自回归图像生成**（GPT-Image 路线）的核心前提。对扩散范式而言，ViT 提供了架构优化；对自回归范式而言，ViT 提供了存在前提。

CNN & ViT 参考文档

<https://www.datacamp.com/tutorial/vision-transformers> (VIT)

<https://www.codecademy.com/article/vision-transformers-working-architecture-explained> (VIT)

<https://learnopencv.com/understanding-convolutional-neural-networks-cnn/> (CNN)

<https://www.codecademy.com/article/understanding-convolutional-neural-network-cnn-architecture> (CNN)

2. CLIP (2021)

CLIP — Contrastive Language-Image Pre-training (2021)

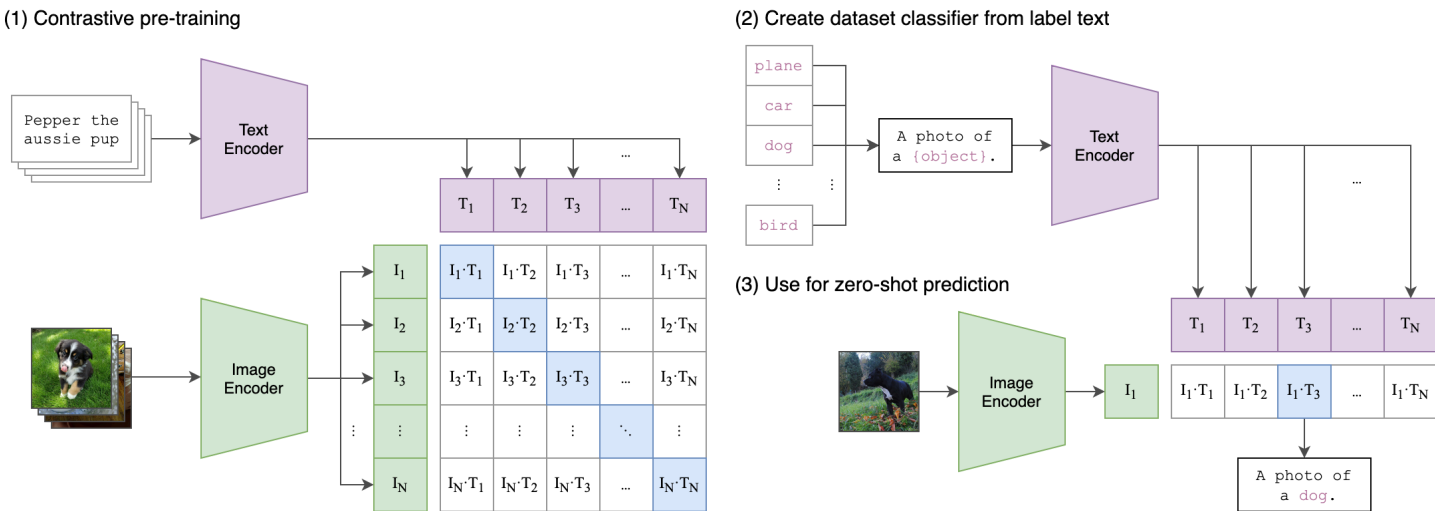
Learning Transferable Visual Models From Natural Language Supervision — Radford et al. (OpenAI) (2021)<https://arxiv.org/abs/2103.00020>

 实现了图文的跨模态对齐。

简介

由 **OpenAI** 提出，是一个通过**对比学习**将**图像和文本**映射到同一语义空间的模型。

它使用了从互联网收集的 4 亿个图像-文本对，训练一个**图像编码器**（ResNet 或 ViT）和一个**文本编码器**（Transformer），使匹配的图文对在嵌入空间中距离更近，不匹配的更远。



CLIP 架构图。把一批图片和对应的文字描述分别扔进两个编码器，得到两组向量，然后算它们两两之间的相似度，配对的拉近，不配对的推远，蓝色对角线就是训练目标。右边是使用阶段：把候选类别（plane、car、dog、bird）套进模板变成文字向量，再把待分类的图片变成图片向量，哪个文字向量离图片向量最近，那个类别就是答案，全程不需要重新训练（Zero-shot）

CLIP 的训练

- 训练数据是从互联网上爬取的 4 亿个图文对（图片 + 对应的文字描述）。

- 模型有两个编码器：一个图像编码器（ResNet 或 ViT），一个文本编码器（Transformer）。每个编码器把各自的输入压缩成一个固定长度的向量（比如 512 维）。
- 训练目标就是对比学习：一个 batch 里有 N 个图文对，正确配对的图文向量要拉近（余弦相似度 $\rightarrow 1$ ），其余 $N^2 - N$ 个错误配对要推远（余弦相似度 $\rightarrow 0$ ）。训练完成后，两个编码器就学会了把语义相同的图和文映射到同一张地图上的相邻位置。

CLIP 的使用（文生图场景）

- CLIP 训练完之后，在文生图场景中只用它的文本编码器。
- 流程：用户输入 prompt $\rightarrow$ CLIP 文本编码器将其编码为一个 512 维向量 $\rightarrow$ 这个向量作为条件信号注入生成模型（如扩散模型），在每一步去噪过程中引导生成方向。
- 注意：CLIP 本身不生成图像，也不担任生图模型的核心网络结构。它是一个工具，只负责"把人话翻译成生成模型能理解的坐标"，是文生图 pipeline 中的语义翻译层。

核心贡献

- 实现了文本与图像之间的大规模语义对齐，成为所有「文本到图像」系统的语义基础设施。
- 之前的 GAN 和 VAE 只能做无条件生成或粗粒度类别控制，CLIP 让「用自然语言描述来引导生成」成为可能。

局限性

- 组合理解能力弱：CLIP 对属性绑定（"黄色的小猫的杯子和白色的花"vs"白色的小猫和红色的花"）和空间关系的理解较弱，基于 CLIP 引导的生成模型也继承了这一缺陷。

地位 & 影响

CLIP 是连接视觉和语言两大模态的桥梁，它的出现深刻改变了文本到图像生成的研究路径。

在扩散范式中，**Stable Diffusion** 使用 CLIP 的文本编码器作为条件输入。**DALL·E 2** / unCLIP 则以 CLIP 为核心构建。

CLIP 参考文档

<https://openai.com/index/clip/>（推荐）

<https://viso.ai/deep-learning/clip-machine-learning/>

<https://medium.com/@ManishChablani/clip-contrastive-language-image-pretraining-summary-and-intuition-52e329a67377>

三、扩散范式

扩散模型的核心思想是：前向过程逐步向图像加高斯噪声直到变成纯噪声，反向过程训练一个网络逐步去噪。破坏容易，重建难，但如果把破坏过程拆成足够多的小步，每一步的逆过程就变得可学习。

1. DDPM (2020)

Denoising Diffusion Probabilistic Models — Ho, Jain & Abbeel

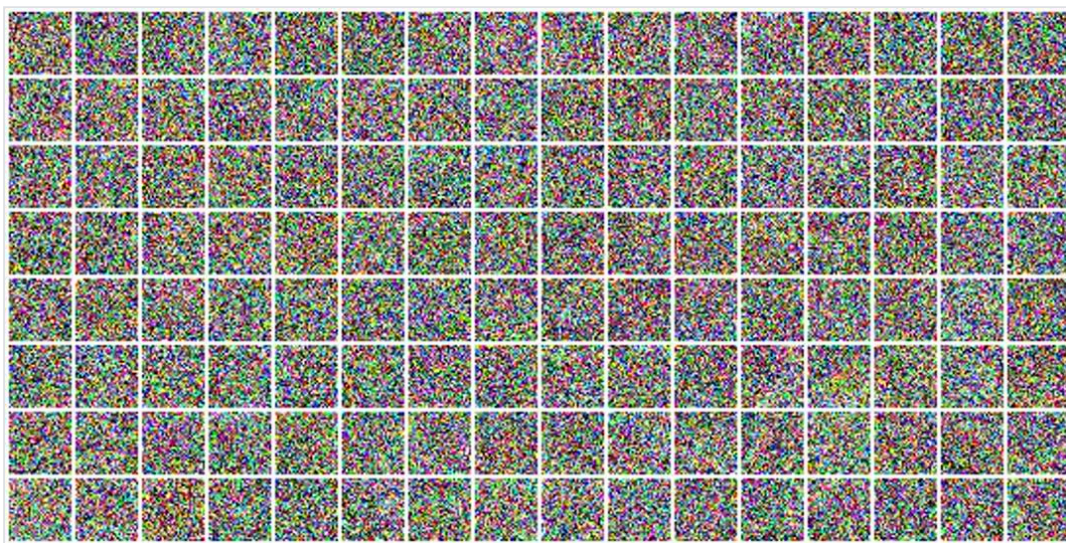
(2020)<https://arxiv.org/abs/2006.11239>



扩散模型时代的开端；训练时先「加噪」再「去噪」。

简介

扩散概率模型的思想最早可追溯到 Sohl-Dickstein et al. (2015)，但真正让扩散模型在生成质量上“能用”的是 DDPM。其核心思想优雅而直觉化：前向过程逐步向数据添加高斯噪声，直至变为纯噪声；反向过程训练一个神经网络（**U-Net**）逐步去噪，从纯噪声恢复出数据。训练目标被简化为：**让网络预测每一步添加的噪声**（即简化的变分下界）。



DDPM 训练过程

• 前向过程：加噪

- 给定一张真图 x_0 ，按照预设的噪声调度表（noise schedule），每一步叠加一点高斯噪声。
- 经过 T 步（通常 $T=1000$ ）后，原图信息完全被淹没，变为纯高斯噪声。
- 前向过程是纯数学操作，不涉及任何可学习参数。一个关键性质是：给定任意时间步 t ，可以用封闭公式直接从 x_0 一步算出 x_t ，无需逐步迭代，这对**训练效率**至关重要。

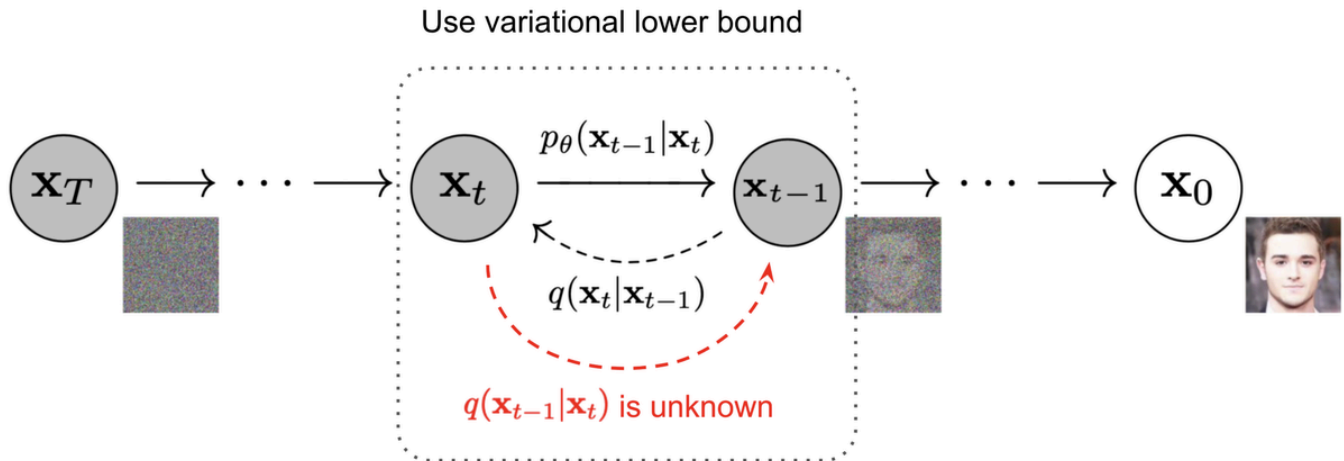
• 反向过程：去噪

- 训练一个去噪网络，输入为**加噪后的图像 x_t** 和**当前时间步 t** ，输出为对该步所加噪声的预测。
- 训练流程：每轮从训练集随机抽取一张真图，随机采样一个时间步 t ，用封闭公式直接计算 x_t ，然后让网络预测所加噪声，与真实噪声做 MSE 对比，反向传播更新参数。没有对抗、没有博弈，就是一个朴素的回归任务。

- 时间步 t 也作为条件输入喂给网络，是因为不同阶段的去噪策略截然不同：早期步骤（ t 大，图几乎是纯噪声）侧重生成大的结构和轮廓；后期步骤（ t 小，图已基本成型）侧重补充纹理和高频细节。网络需要知道“我现在处理的是哪个阶段”。

- 损失函数：

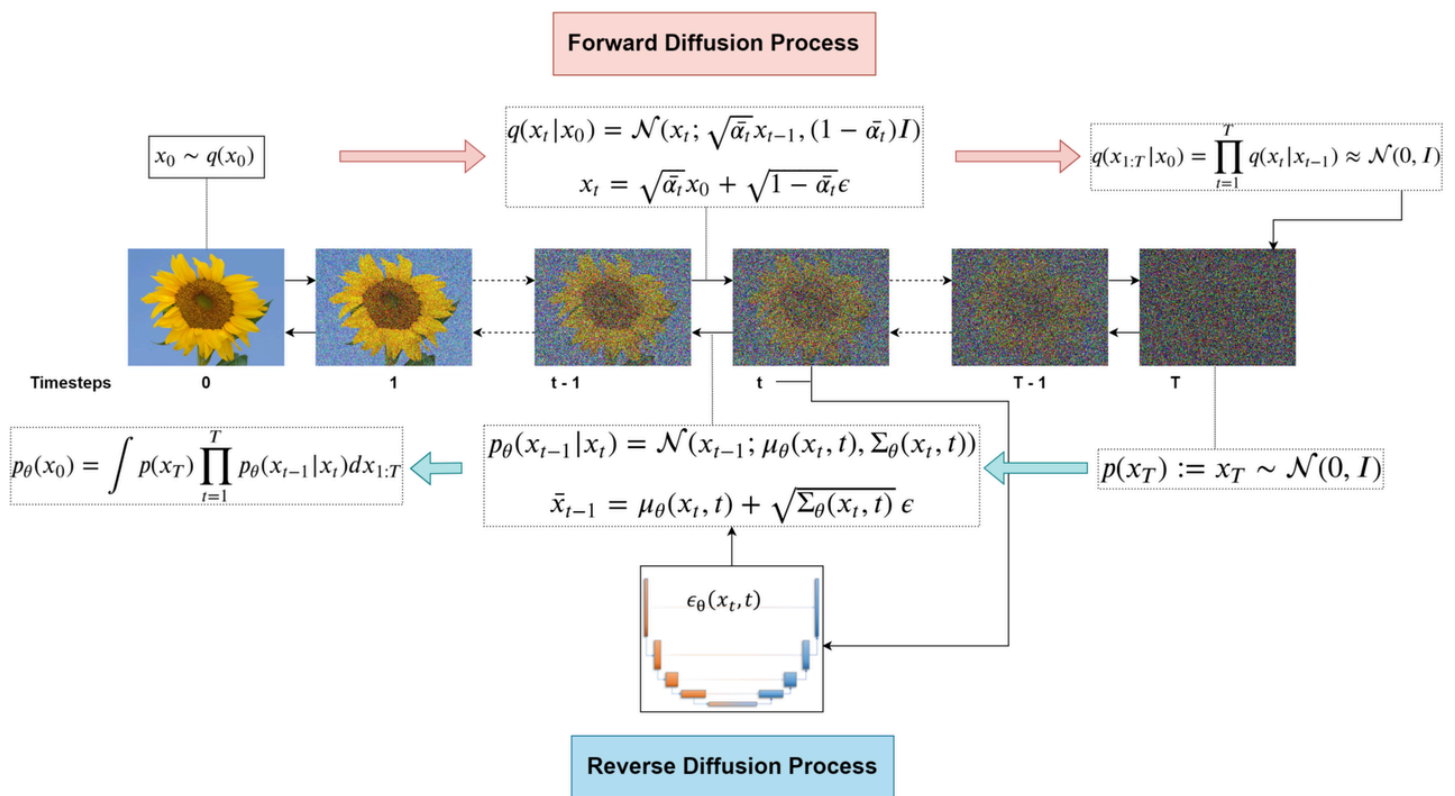
- MSE 均方误差，表示真实噪声和预测噪声之间的差距。



通过缓慢添加（删除）噪声来生成样本的正向（反向）扩散过程的马尔科夫链。上面的黑色箭头 $p_{\theta}(x_{t-1}|x_t)$ 是我们训练的神经网络要学的东西：给定当前的噪声图 x_t ，预测去掉一步噪声后的 x_{t-1} 应该长什么样的函数

DDPM 生图过程

- 训练完成后，生成图像的流程为：随机采样一张纯高斯噪声图作为 x_T ，从 $t=T$ 开始，每一步让去噪网络预测当前噪声并将其减去，同时注入一点更小的随机噪声（维持采样的随机性），得到 x_{t-1} ；
- 如此反复迭代 T 步，噪声图中逐渐浮现出结构、轮廓、纹理，最终得到一张完整的生成图像 x_0 。不同的初始噪声会生成不同的图像。



DDPM 的完整流程，上半部分（红色）：Forward Diffusion Process — 前向加噪。 $q(x_t|x_0)$ 这个公式表示你可以从原图 x_0 直接一步跳到任意时间步 t 的噪声图，不用真的一步步加；右上角的公式说整个前向过程是一条马尔可夫链，每一步的加噪只依赖上一步，走到最后 T 步时，结果近似于标准高斯分布 $\mathcal{N}(0, I)$ ；下半部分（蓝色）：Reverse Diffusion Process — 反向去噪。每一步的去噪操作是：把当前噪声图 x_t 和时间步 t 丢给去噪网络 $\epsilon_\theta(x_t, t)$ （最底部那个 U-Net 结构的示意图）网络预测出当前的噪声。然后用预测结果算出 x_{t-1} 的均值 μ_θ 和方差 Σ_θ ，再加上一

点随机噪声 ϵ 采样得到 x_{t-1} 。公式就是 $x_{t-1} = \mu_\theta(x_t, t) + \sqrt{\Sigma_\theta(x_t, t)} \cdot \epsilon$ 。

去噪网络架构

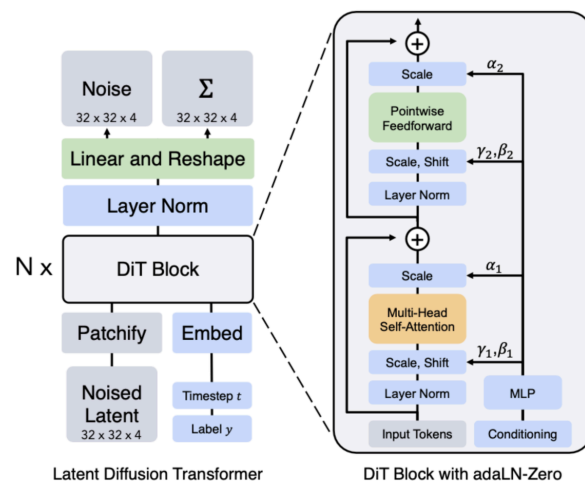
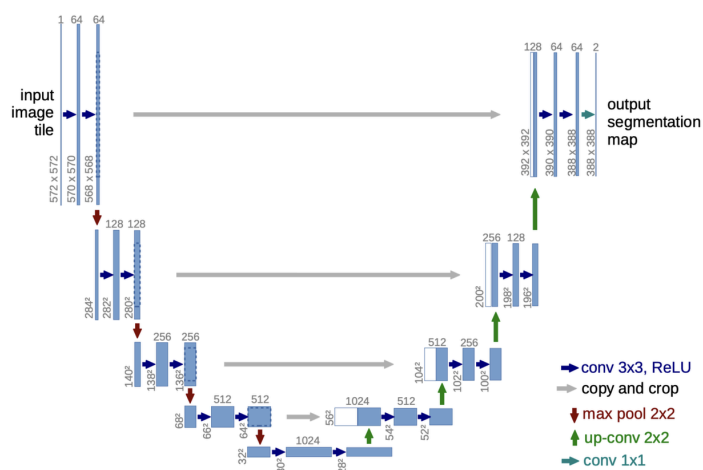
扩散模型的去噪网络有两种主流架构选择：

- **U-Net**：DDPM 原论文采用的架构。对称的编码器-解码器结构，编码器逐层压缩提取抽象特征，解码器逐层放大还原分辨率，两侧通过跳跃连接（skip connections）直接传递信息，使输出同时保留深层语义和浅层细节。输入输出尺寸相同，天然适合“输入噪声图、输出预测噪声”的去噪任务。从 DDPM 到 Stable Diffusion，U-Net 一直是扩散模型的默认骨干。

<https://arxiv.org/abs/1505.04597>

- **Transformer (DiT)**：后来 **DiT (Diffusion Transformer)** 将骨干替换为 Transformer——把噪声图像切成 patch 序列，用注意力机制让每个 patch 从第一层就能与所有其他 patch 直接交互，全局信息捕捉效率更高，且扩展性（scaling）更优。**Sora** 等新一代模型已采用此架构。

<https://arxiv.org/abs/2212.09748>



Diffusion Transformer (DiT) 架构

核心贡献

- **简化的训练目标：** 将复杂的变分下界分解为每个时间步的去噪损失，最终等价于一个简单的 MSE 噪声预测目标。这使得训练变得极为稳定和直观，与训练 GAN 的复杂性形成鲜明对比。
- **高质量生成：** 在 CIFAR-10 和 LSUN 等数据集上首次展示了扩散模型可以生成与 GAN 媲美甚至更优质的图像，且不存在模式坍塌问题。
- **理论优雅性：** 建立了前向扩散和反向去噪之间的严格数学对应关系，使得模型具有清晰的概率解释。

局限

- **采样速度慢：** 反向去噪需要上千步迭代（如 $T=1000$ ），每一步都需要一次完整的网络前向传播，生成一张图像需要数分钟，远慢于 GAN 的单次前向传播。
- **分辨率受限：** 在像素空间直接做扩散的计算成本随分辨率平方增长，DDPM 的实验仅限于 32×32 和 256×256 。
- **无条件生成：** 原始 DDPM 仅做**无条件生成**，不支持文本、类别等条件引导。

地位 & 影响

DDPM 是扩散模型时代的真正开端。后续所有扩散模型（Stable Diffusion、DALL·E 2、Midjourney 等）都建立在 DDPM 的框架之上。

DDPM 参考文档

<https://lilianweng.github.io/posts/2021-07-11-diffusion-models/>（非常严谨清晰，推荐）

<https://learnopencv.com/denoising-diffusion-probabilistic-models/>

<https://theaisummer.com/diffusion-models/>

<https://zhouyifan.net/2023/07/07/20230330-diffusion-model/>（中文）

<https://calvinluo.com/2022/08/26/diffusion-tutorial.html>

2. Diffusion Models Beat GANs (2021)

Diffusion Models Beat GANs on Image Synthesis — Dhariwal & Nichol (OpenAI)
(2021) <https://arxiv.org/abs/2105.05233>



通过「分类器」实现了「条件化生图」；是扩散正式超越 GAN 的转折点。

简介

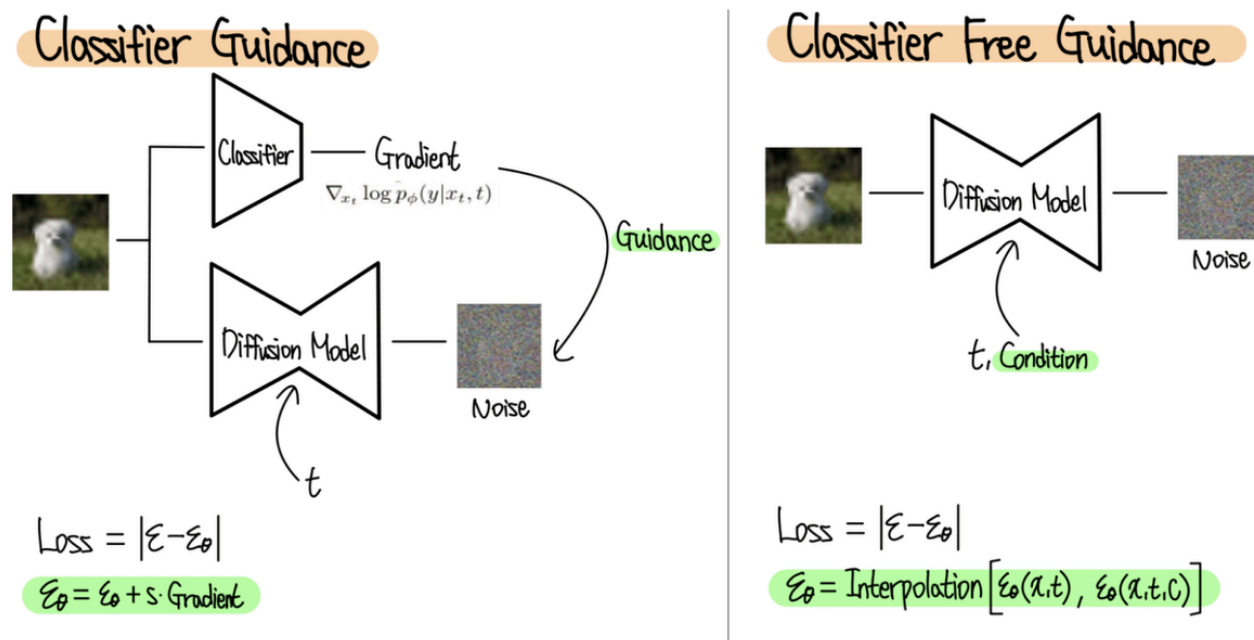
这篇 OpenAI 的工作直面一个核心问题：扩散模型在理论上具有诸多优势，但在生成质量指标（FID）上仍落后于最好的 GAN（BigGAN、StyleGAN）。作者系统性地改进了 DDPM 的架构（更大的 U-Net、自适应归一化、更多注意力层），**首次让扩散模型在 ImageNet 条件生成上超越了 GAN。**

核心贡献

- **架构改进：**对 U-Net 进行了全面升级——增加网络深度和宽度、引入多头自注意力、使用 Adaptive Group Normalization 注入时间步信息、增大注意力头数等，系统性地提升了模型容量。
- **分类器引导（Classifier Guidance）：**这是**本文最重要的创新**。在采样时用一个在噪声数据上预训练分类器的梯度来引导去噪方向，大幅提升图像质量和条件控制能力。

代码块

- 1 具体来说，每一步去噪时：
- 2 1. 扩散模型先正常预测去噪方向
- 3 2. 把当前的中间结果丢给分类器，问它"这像不像目标类别（比如'金毛犬'）？"
- 4 3. 分类器算出一个梯度，指示"往哪个方向改一改就更像金毛犬"
- 5 4. 把这个梯度叠加到去噪方向上（最终方向 = 原始去噪方向 + 分类器梯度）
- 6
- 7 注意这里的关键：**扩散模型本身完全不认识文字，也没有任何文本输入**。它就是个无条件去噪器。所有的"引导"都来自外部的分类器梯度硬拽。



Classifier Guidance & Classifier Free Guidance

- **FID 首超 GAN：** 在 ImageNet 256×256 和 512×512 上，扩散模型首次在 FID 指标上击败 BigGAN-deep，标志着范式转折。

局限性

- **需要额外训练分类器：** 分类器引导要求在噪声数据上训练一个独立的分类器，增加了训练成本和流程复杂度。
- **仅支持类别条件：** 分类器引导天然适合类别标签，但难以扩展到自由文本等更灵活的条件。
- **采样仍然较慢：** 虽然在质量上超越了 GAN，但采样速度仍然是 GAN 的百倍以上。

地位 & 影响

这篇论文是扩散范式崛起的标志性事件，它在实证上终结了 GAN 的统治地位。

Classifier guidance 的思想后来被 classifier-free guidance 替代（不需要额外分类器，直接在训练时随机 drop 条件），成为几乎所有条件生成模型（GLIDE、DALL·E 2、Stable Diffusion、Imagen）的标准做法。

3. GLIDE (2021)

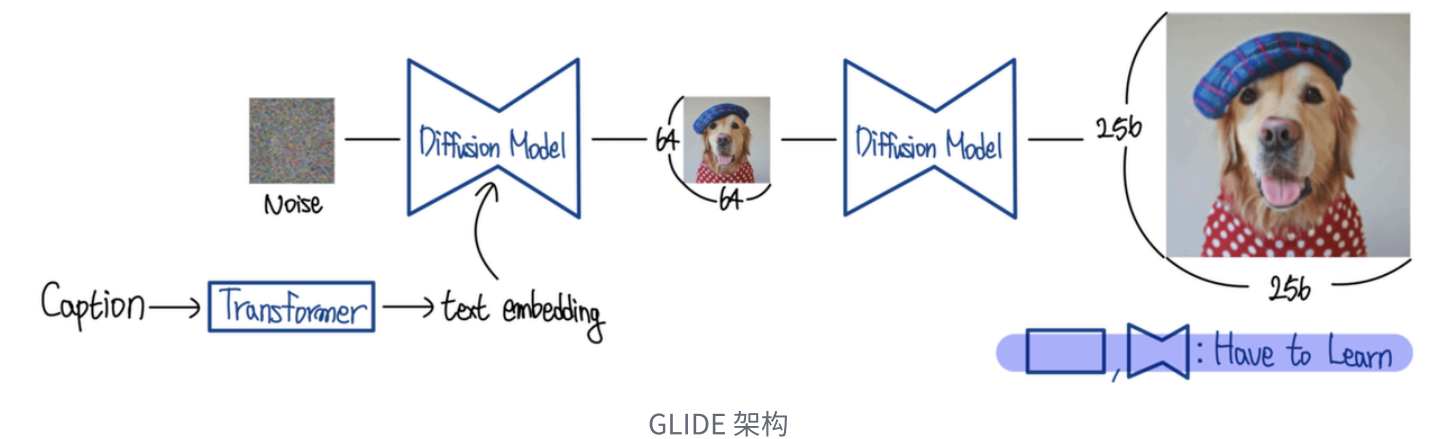
GLIDE - Guided Language to Image Diffusion for Generation and Editing

GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models — Nichol et al. (OpenAI) (2021) <https://arxiv.org/abs/2112.10741>

💡 实现了无需「分类器」的「自然语言生图」。

简介

GLIDE 是 OpenAI 探索**文本引导扩散模型**的重要里程碑。在它之前，Beat GANs 提出的 Classifier Guidance 虽然能引导生成，但依赖一个在噪声数据上额外训练的分类器，且天然只适配类别标签，难以扩展到自由文本。GLIDE 系统性地验证了一种更优雅的替代方案——**Classifier-Free Guidance (CFG)**，无需任何外部分类器，引导能力完全内化于扩散模型自身，并将其与 cross-attention 文本注入结合，确立了文本引导扩散模型的标准范式。



网络结构

GLIDE 的主体是一个扩散模型，骨干为 **U-Net**。文本部分由一个 Transformer 文本编码器处理：将 prompt 编码为一串 token 嵌入，再通过 cross-attention 注入 U-Net 的各层，使去噪网络在每一步都能"读到"完整的文本语义，而非仅在起始处接收一次条件信号。整个模型规模为 35 亿参数；生成在 64×64 像素空间进行，再由一个独立的上采样扩散模型提升至 256×256 。

训练过程

- GLIDE 的训练在标准扩散模型（DDPM）的去噪目标之上，增加了**文本条件与 Classifier-Free Guidance** 的关键设计。需要强调的是，**整个过程只训练一个去噪网络，不涉及任何额外的分类器。**
- 输入构成：网络的输入由三部分组成——**加噪图像 x_t 、时间步 t 、以及经文本编码器编码后的文本嵌入**。任务仍是预测该步所加的噪声，损失函数为预测噪声与真实噪声之间的 MSE。
 - **条件随机丢弃 (CFG 的核心)**：训练时以一定比例（约 10%–20%）随机将文本条件替换为一个空的占位符，让网络在"无文本"状态下完成去噪。如此一来，同一个网络同时学会了两种模式——**有文本条件的去噪与无条件的去噪**。这正是 "Classifier-Free" 的由来：引导能力不再来自外部分类器，而是由网络自身的"有条件 / 无条件"两种预测之差提供。
 - cross-attention 文本注入：文本嵌入通过 cross-attention 参与 U-Net 每一层的计算，使文本语义在整个去噪过程中持续发挥引导作用。

生成过程

训练完成后，GLIDE 通过 Classifier-Free Guidance 实现高质量的文本引导采样。

- **双路预测**：在每一个去噪步骤，让同一个网络对当前噪声图做两次预测——**一次输入文本嵌入，得到有条件预测；一次输入空占位符，得到无条件预测。**

- 引导放大：两次预测之差代表"文本条件额外施加的方向"，最终的去噪方向由无条件预测加上该差值的若干倍构成，即「无条件方向 + 引导强度 × (有条件方向 - 无条件方向)」。引导强度越大，生成结果越贴合文本描述，但过大会牺牲多样性与真实感。
- 迭代与上采样：如此从纯噪声开始反复迭代，逐步浮现出符合文本的 64×64 图像，再经上采样扩散模型还原为 256×256 的最终结果。

核心贡献

- **Classifier-Free Guidance 的实践验证**：GLIDE 系统地展示了 **Classifier-Free Guidance** (CFG) 在文生图任务上优于 CLIP 引导——生成的图像更逼真、更符合文本描述。CFG 无需额外分类器或 CLIP 模型，直接在扩散模型内部完成引导。
- **文本到图像的高质量扩散**：35 亿参数的 GLIDE 模型在人类评估中被认为优于 **DALL·E 1**（基于自回归的 120 亿参数模型），首次证明了扩散模型在文生图领域的竞争力。
- **图像编辑能力**：GLIDE 支持基于文本描述的 inpainting——用户可以遮盖图像的一部分，用文本描述要填充的内容，模型会生成语义一致的补全。

局限性

- **仍在像素空间操作**：GLIDE 直接在 64×64 像素空间做扩散，再用上采样扩散模型提升到 256×256 ，计算成本高昂。
- **文本理解仍有限**：对复杂的组合语义和空间关系的理解不够精确。

地位 & 影响

GLIDE 处于从"扩散模型超越 GAN"到"**扩散模型主导文生图**"的关键过渡期。

它确立了 **Classifier-Free Guidance** 作为文本引导扩散模型的标准方法，这一技术被后续的 **DALL·E 2**、**Imagen**、**Stable Diffusion** 等全面采纳。

GLIDE 也直接启发了 DALL·E 2 的设计——后者在 GLIDE 的基础上引入 CLIP 嵌入作为中间表示，进一步提升了文本-图像的对齐质量。

GLIDE 参考文档

<https://ffighting.net/deep-learning-paper-review/diffusion-model/glides/>

4. LDM (2021) / Stable Diffusion

High-Resolution Image Synthesis with Latent Diffusion Models — Rombach et al.
(2021)<https://arxiv.org/abs/2112.10752>



引入 VAE，在隐空间内生图，解决扩散架构的算力问题。

简介

潜在扩散模型（Latent Diffusion Model, LDM）的核心洞察是：在像素空间直接做扩散既昂贵又浪费——图像的高频细节对语义信息贡献有限，却消耗了大量计算。

LDM 的解决方案是**将扩散过程从像素空间搬到一个预训练 VAE 的潜在空间中**：先用 VAE 的编码器将图像压缩为低分辨率的隐表示（例如从 512×512 压缩到 64×64 ），在隐空间中训练扩散模型，最后用 VAE 的解码器将去噪后的隐表示还原为高分辨率图像。这一架构后来被 Stability AI 用于训练并开源 **Stable Diffusion**，成为整个 AI 生图大众化浪潮的技术起点。

模型结构

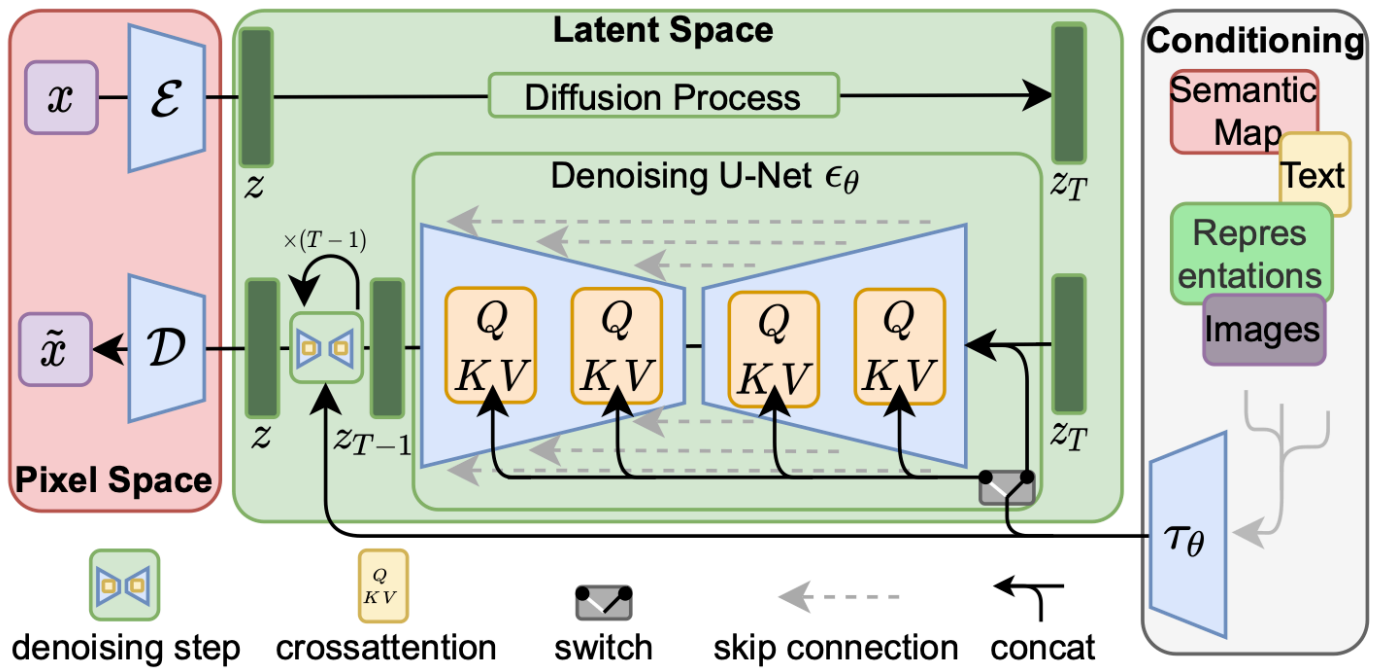
LDM 由三个相对独立的组件构成，三者各司其职、可分别优化：

- **VAE（编码器 + 解码器）**：负责在像素空间与潜空间之间转换。编码器将高分辨率图像压缩为低维隐表示（如 $512 \times 512 \rightarrow 64 \times 64$ ），解码器负责将隐表示还原回高分辨率图像。VAE 在扩散训练之前已单独训练完成，扩散阶段其参数固定不动。
- **去噪 U-Net**：扩散模型的骨干，但它作用的对象不再是像素图，而是 VAE 输出的隐表示。结构与 DDPM 的 U-Net 一致，输入输出尺寸相同，任务仍是预测噪声。
- **条件机制（cross-attention）**：继承 GLIDE 的思路，文本由一个文本编码器编码为嵌入，再通过 cross-attention 注入 U-Net 的各层，引导生成方向。除文本外，图像布局、语义图等条件也可通过类似方式接入。（注：Rombach 等人的原始 LDM 论文做文生图时采用的是 BERT 风格的 Transformer 文本编码器；后来 Stability AI 训练 Stable Diffusion v1 时才换用 CLIP ViT-L/14 的文本编码器）

训练过程

LDM 的训练分为两个阶段，前一阶段为后一阶段提供压缩空间。

- **第一阶段：训练 VAE**：在大规模图像数据上训练 VAE，使编码器学会把图像压缩为紧凑的隐表示、解码器能高质量地将其还原。该阶段独立完成，目标是得到一个“感知压缩”良好的隐空间。训练完成后 VAE 被冻结。
- **第二阶段：在隐空间训练扩散模型**：取一张真图，先用冻结的 VAE 编码器将其压缩为隐表示 z_0 ；随后对 z_0 执行与 DDPM 完全相同的扩散流程——按时间步 t 向 z_0 加噪得到 z_t ，让 U-Net 预测所加噪声，与真实噪声做 MSE 并反向传播。文本条件经编码后通过 cross-attention 参与每一步预测。关键在于：所有加噪、去噪、迭代都发生在低维隐空间，网络处理的数据量比像素空间小一到两个数量级，训练成本大幅下降。



LDM 架构解释

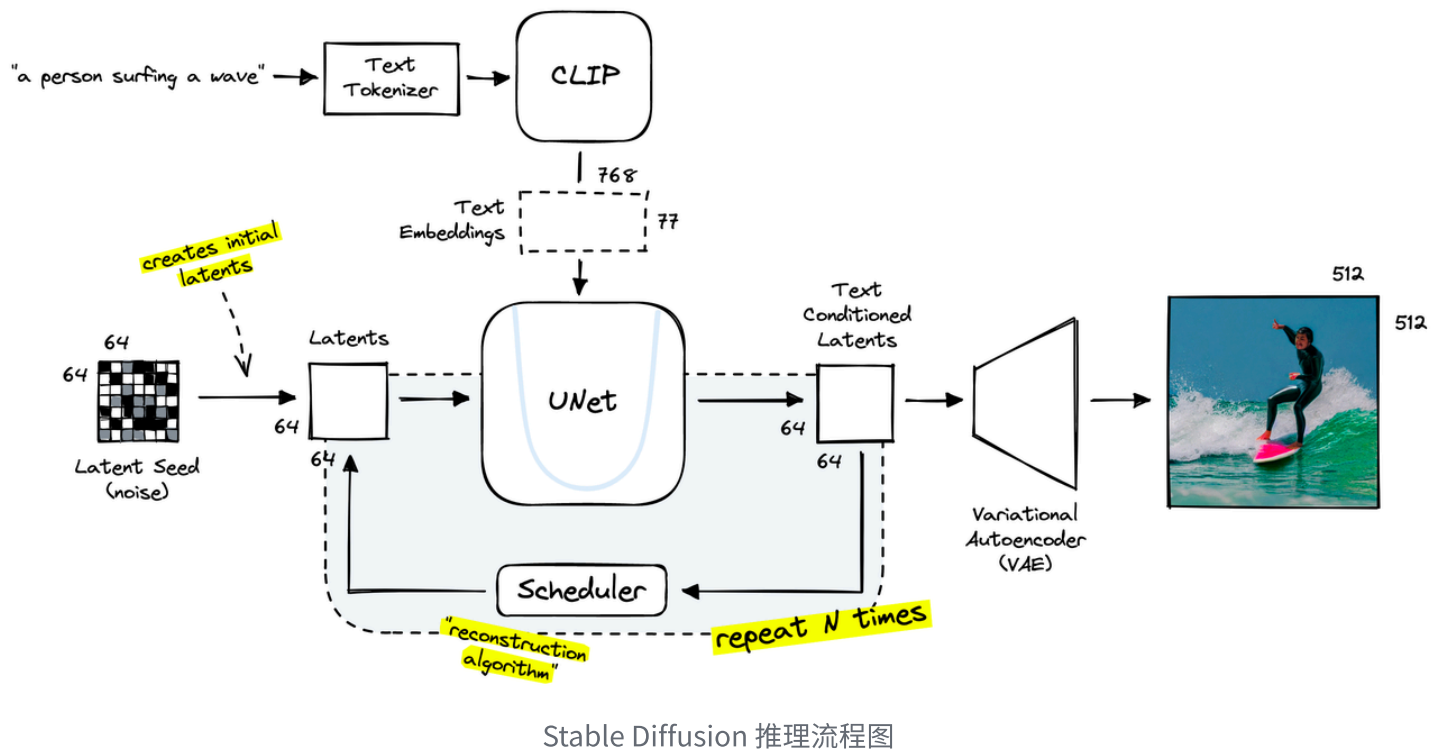
- 1 左边的红色区域是 Pixel Space（像素空间）。
- 2 原始图像 x 在这里进出。上方的编码器 E 把图像压缩成 latent z ，下方的解码器 D 把 latent 还原回图像 $\tilde{x}$ 。这一对 E/D 就是一个预训练好的 autoencoder（论文里用的是带感知损失的 VQ-VAE 变体），训练完就冻住不动了。它的作用只有一个：让后面所有的扩散运算都在小得多的 latent 上进行，而不是在高分辨率像素上硬算。
- 3
- 4 中间绿色区域是 Latent Space（潜空间）。
- 5 这是整个模型真正干活的地方。上方那条从左到右的箭头标着 "Diffusion Process"，就是前向加噪：干净的 latent z 被逐步加噪变成纯噪声 z_T 。下方那个蓝色的蝴蝶结形状就是 Denoising U-Net ϵ_θ ，负责反向去噪：从 z_T 一步步预测噪声、还原出 z 。上面标的 $\times(T-1)$ 表示这个去噪过程要迭代 T 步。
- 6
- 7 右边的灰色区域是 Conditioning（条件输入）。
- 8 这里画了好几种可能的条件来源：语义图（Semantic Map）、文本（Text）、图像（Images）等等。不管输入是什么模态，都先通过一个领域专用的编码器 τ_θ 编码成统一的中间表示（Representations），然后送进 U-Net。

生成过程

训练完成后，LDM 的采样流程在隐空间完成，仅在最后一步回到像素空间。

- 隐空间采样：在 64×64 的隐空间中随机采样一个纯噪声隐表示 z_T ，从 $t=T$ 开始，让 U-Net 在文本条件（经 cross-attention 注入）的引导下逐步预测并去除噪声，反复迭代 T 步，得到一个干净的隐表示 z_0 。这一过程通常配合 Classifier-Free Guidance 以增强文本贴合度。
- 解码还原：将去噪得到的隐表示 z_0 送入 VAE 解码器，一步还原为高分辨率的最终图像。

由于全部迭代都在小尺寸的隐空间进行，仅最后一次解码涉及高分辨率运算，整体计算开销远低于像素空间扩散，使得高分辨率图像生成在消费级硬件上也变得可行。



Stable Diffusion 推理流程图解释

- 1 上方：文本怎么变成条件信号
- 2 用户输入的提示词 "a person surfing a wave" 先经过 Text Tokenizer 切成 token，再送进 CLIP 文本编码器，输出一个 77×768 的矩阵——77 是 token 序列长度（固定填充），768 是每个 token 的嵌入维度。这个矩阵就是后面 U-Net 里 cross-attention 的 Key 和 Value。
- 3
- 4 左侧：起点是一张纯噪声
- 5 一个 64×64×4 的随机噪声张量作为 Latent Seed。注意不是 512×512 的像素图，而是压缩了 8 倍的潜空间表示，这就是 "Latent" Diffusion 省算力的关键。
- 6
- 7 中间：U-Net + Scheduler 的迭代去噪循环
- 8 这是整张图的核心。每一轮迭代里，U-Net 同时接收当前的 noisy latent 和文本 embedding，预测出噪声；Scheduler（黄色标注的 "reconstruction algorithm"）根据这个预测决定怎么更新 latent——减去多少噪声、保留多少信号。然后把更新后的 latent 送回 U-Net 再来一轮。这个循环重复 N 次（通常 20~50 步），latent 从纯噪声逐渐变成干净的、符合文本描述的图像表示。
- 9
- 10 右侧：VAE 解码器还原成像素
- 11 循环结束后，64×64 的去噪 latent 送进 VAE 解码器，一次性放大并还原为 512×512 的像素图像。

核心贡献

- 计算效率的质变：把计算量降了一到两个数量级，让高分辨率图像生成变得实际可行，这是 Stable Diffusion 能够被广泛部署和开源的技术基础。

- 跨范式结合：这篇论文是 VAE 和 DDPM 的优美结合——VAE 负责压缩/解码，DDPM 在压缩空间里做生成。感知压缩（VAE）和生成建模（扩散）分离，各自可以独立优化，并且可以灵活地接入各种条件机制（文本通过交叉注意力、图像布局通过拼接等）。
- Cross-Attention 条件机制：继承了 GLIDE 的思路，将文本条件通过 cross-attention 注入 U-Net，成为文本条件扩散模型的标准实现方式。
- 开源与社区生态：Stability AI 基于 LDM 训练并开源了 Stable Diffusion，引发了 AI 生图的大众化浪潮，催生了 ControlNet、LoRA 微调、各种社区模型等庞大生态。

局限性

- VAE 重建瓶颈：图像质量的上限受到 VAE 解码器的限制——VAE 会丢失部分高频细节，导致生成图像在精细纹理上可能不如像素级扩散模型。
- 两阶段训练：需要先训练好 VAE，再训练扩散模型，增加了流程复杂度。
- 小物体和文字生成弱：隐空间的压缩使得模型在生成精细文字、小物体等方面表现不佳。

地位 & 影响

这篇论文就是 **Stable Diffusion** 的学术基础。它的开源让整个社区爆发，直接催生了 **Midjourney** 等生态。可以说是整个扩散生成领域影响力最大的一篇文章。

Stable Diffusion 参考文档

<https://jalammr.github.io/illustrated-stable-diffusion/>

<https://www.louisbouchard.ai/latent-diffusion-models/>

5. DALL·E 2 (2022)

Hierarchical Text-Conditional Image Generation with CLIP Latents — Ramesh et al. (OpenAI)
(2022) <https://arxiv.org/abs/2204.06125>



通过「反转 CLIP」的创意来生图，提升扩散架构图文对齐质量。

简介

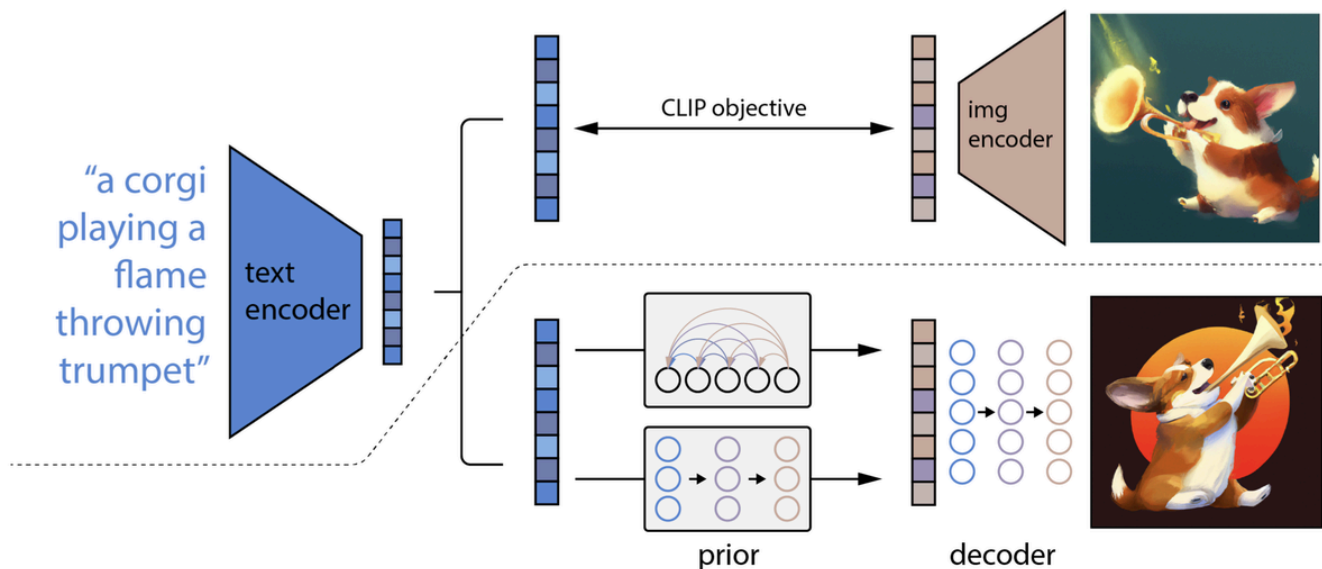
DALL·E 2 是 OpenAI 在 DALL·E 1 之后推出的第二代文生图模型，但技术路线发生了根本转变——**从自回归范式转向了扩散范式**。

它的核心创意是“**反转 CLIP**”：先用 CLIP 文本编码器将文本映射为文本嵌入，然后训练一个“Prior”模型（扩散模型或自回归模型）将文本嵌入映射为 CLIP 图像嵌入，最后用一个扩散解码器从 CLIP 图像嵌入生成图像。整个生成过程在 CLIP 的联合语义空间中展开，因而天然继承了 CLIP 强大的图文对齐能力。

模型结构

DALL·E 2（论文中也称 unCLIP）由三个核心组件串联而成，外加一个上采样模块

- **CLIP:** 提供整个系统所依赖的图文联合嵌入空间,本身是一个预训练完毕并冻结的**双塔对比学习模型**——文本塔(**Transformer**)与图像塔(**ViT**)各自独立编码,通过对比学习把图、文对齐到同一空间。其文本编码器负责把 prompt 编码为文本嵌入;图像编码器仅在训练阶段使用,为训练图像生成对应的 CLIP 图像嵌入,作为 Prior 的监督目标。
- **Prior:** 负责"文本语义 → 视觉语义"的映射,即把 CLIP 文本嵌入转换为 CLIP 图像嵌入。论文尝试了自回归与扩散两种实现,最终采用效果更佳的扩散版;其骨干是 **Transformer** 而非 U-Net。
- **Decoder:** 负责"视觉语义 → 像素"的生成,是一个以 CLIP 图像嵌入为条件的**扩散模型(U-Net 骨干,改编自 GLIDE)**,将图像嵌入还原为图像,再经额外的上采样扩散模型提升至高分辨率。
- **Upsampler**（上采样模型）: Decoder 先生成较低分辨率的图像(64×64),再经一至两级上采样扩散模型逐步提升至 1024×1024 ($64 \rightarrow 256 \rightarrow 1024$)。它同样是**扩散模型(U-Net 骨干)**,但任务不是"从图像嵌入无中生有",而是以低分辨率图作为条件,通过扩散去噪"生成"出放大后的高频细节,因此能避免简单插值放大导致的模糊。这部分只负责提升清晰度,不改变图像的语义内容。



DALL·E 2 总览图

DALL·E 2 总览图解释

1 虚线以上：CLIP 预训练

- 2 文本 "a corgi playing a flame throwing trumpet" 经 text encoder 变成一个文本向量。
- 3 对应的图片经 img encoder 变成一个图像向量。
- 4 CLIP objective（双向箭头）拉近配对的两者、推远不配对的。
- 5 训练完，这两个编码器就冻住不动了，后面全程不再变。

6

7 虚线以下：DALL·E 2 生图过程

- 8 第一步 **prior（先验）**：拿冻住的 text encoder 输出的文本向量，把它"翻译"成一个 CLIP 图像向量。图里画了两种实现方式：上面带弧线箭头的是自回归（autoregressive）prior，下面是扩散（diffusion）prior，论文最后选了扩散版。
- 9 第二步 **decoder（解码器）**：拿 prior 产出的图像向量作为条件，用一个扩散模型生成最终图片。

10

训练过程

DALL·E 2 的训练围绕 Prior 和 Decoder 两个可学习组件展开，CLIP 全程冻结、仅提供嵌入。

- **准备工作（CLIP 冻结）**：复用预训练好的 CLIP。对每个图文对，CLIP 文本编码器给出文本嵌入，CLIP 图像编码器给出对应的图像嵌入——后者将作为 Prior 的训练目标和 Decoder 的训练条件。
- **训练 Prior（文本嵌入 → 图像嵌入）**：输入 CLIP 文本嵌入，让 Prior 预测同一图文对的 CLIP 图像嵌入，以真实图像嵌入为监督目标。如此训练后，Prior 学会了在 CLIP 空间内完成"从文字落点挪到图片落点"的跨越。
- **训练 Decoder（图像嵌入 → 图像）**：这是一个标准的条件扩散模型训练。取一张真图，用 CLIP 图像编码器算出它的图像嵌入作为条件，按 DDPM 的方式对真图加噪，让扩散网络在该图像嵌入的条件下预测所加噪声，以 MSE 反向传播。训练目标是：给定一个 CLIP 图像嵌入，能还原出语义匹配的图像。

生成过程

训练完成后，文本到图像的生成沿"文本 → 文本嵌入 → 图像嵌入 → 图像"的链条逐级展开。

- **文本编码**：用户输入 prompt，经冻结的 CLIP 文本编码器编码为文本嵌入。
- **Prior 映射**：Prior 接收文本嵌入，生成一个对应的 CLIP 图像嵌入——相当于在 CLIP 语义地图上，从"文字所在的位置"跳到"目标图像应在的位置"。
- **Decoder 解码**：扩散解码器以该图像嵌入为条件，从纯噪声开始逐步去噪，生成一张较低分辨率的图像。
- **上采样**：再经上采样扩散模型提升至 1024×1024 ，得到最终结果。

值得一提的是，由于生成必经"CLIP 图像嵌入"这一中间表示，若直接对一张真实图像用 CLIP 图像编码器取嵌入、再交给 Decoder 重新生成，即可得到语义相近但视觉不同的"图像变体"；这也使基于文本的图像编辑变得自然。

核心贡献

- **CLIP 嵌入作为中间表示**：通过在 CLIP 的联合嵌入空间中操作，模型天然继承了 CLIP 强大的语义理解能力，文本-图像的语义对齐质量大幅提升。
- **Prior + Decoder 两阶段架构**：Prior 负责文本语义到视觉语义的映射（文本嵌入 → 图像嵌入），Decoder 负责从视觉语义到像素的生成（图像嵌入 → 图像）。这种分解使得各阶段可以独立优化。
- **图像变体和编辑**：由于生成过程经过 CLIP 图像嵌入，模型天然支持"给定一张图像，生成其语义相似但视觉不同的变体"以及基于文本的图像编辑。
- **产品级质量**：DALL·E 2 是最早引起公众广泛关注的 AI 生图产品之一，其生成质量和多样性震撼了业界。

局限性

- 属性绑定问题：继承了 CLIP 的弱点——在涉及多个对象和属性的组合场景中，容易出现属性混淆（如“红色的立方体和蓝色的球”可能生成颜色错误的搭配）。
- 文字生成能力弱：模型难以在生成的图像中正确渲染文字。
- 多阶段复杂性：需要分别训练 CLIP、Prior、Decoder 和上采样模型，整体流程较为复杂。
- 闭源：DALL·E 2 未开源，限制了学术复现。

地位与影响

DALL·E 2 是当时最强的商业文生图产品；它也标志着文生图领域从自回归范式向扩散范式的转向。

DALL·E 2 与 Google 同期发布的 Imagen (Saharia et al., 2022) 一起，确立了“大规模文本编码器 + 扩散模型”作为文生图的主流架构。

6. DiT (2022)

DiT — Diffusion Transformer (2022)

Scalable Diffusion Models with Transformers — Peebles & Xie
(2022) <https://arxiv.org/abs/2212.09748>



将扩散模型的骨干网络从 U-Net 换成 Transformer。

简介

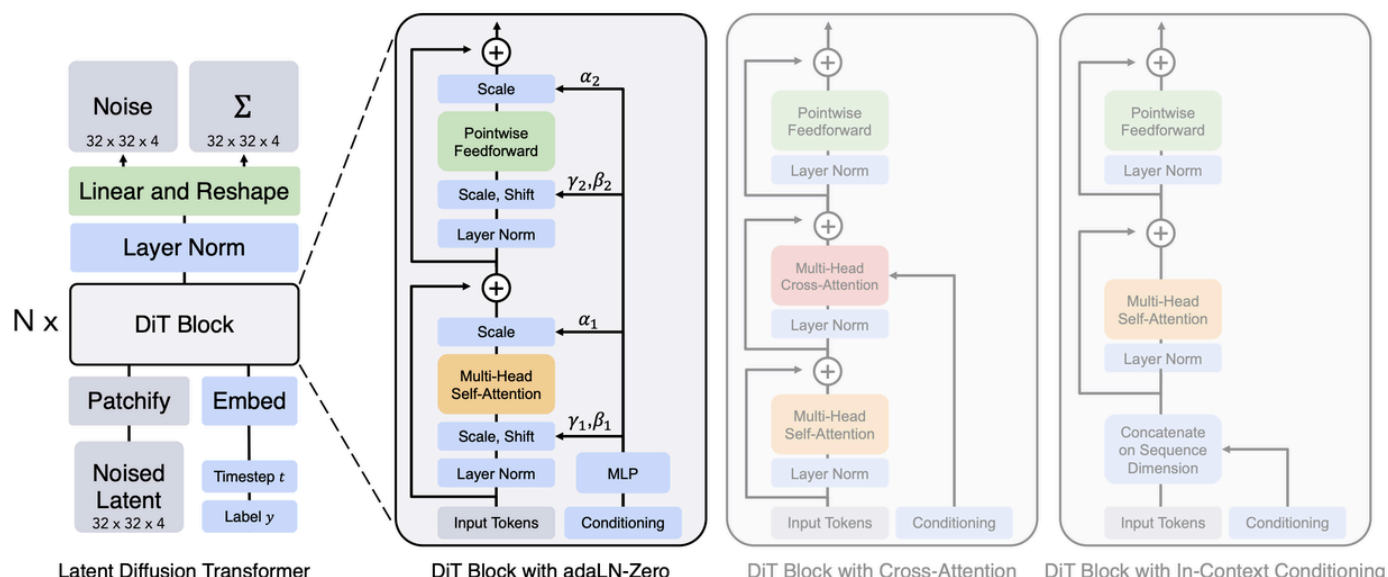
DiT 用 **Transformer** 替换 U-Net 作为扩散模型的骨干网络。在它之前，从 DDPM 到 Stable Diffusion，扩散模型的去噪骨干一直是 U-Net（一种 CNN 架构）；而 ViT 早已证明 Transformer 在视觉任务上不仅可行、且扩展性更优。DiT 正是把这一结论引入扩散范式：将带噪声的潜变量切成 patch 序列（**继承 ViT 的思路**），用标准 Transformer 块处理，并证明扩散模型的性能瓶颈不在 U-Net 架构本身，而在于可扩展性。

模型结构

DiT 是 **ViT**、**LDM**、**Transformer** 三者思想的融合，整体作用在 LDM 式的潜空间中：

- **潜空间输入（来自 LDM）**：DiT 不在像素空间操作，而是先用一个**预训练 VAE** 把图像压缩为低维隐表示，扩散在这个**隐空间**中进行，以降低计算成本。
- **Patch 化与 token 序列（来自 ViT）**：将带噪声的隐表示切分为固定大小的 patch，每个 patch 线性投影为一个 token，从而把二维隐表示转化为一维 token 序列。
- **标准 Transformer 骨干**：用一连串标准 Transformer 块处理该 token 序列，借注意力机制让每个 patch 从第一层起就能与所有其他 patch 直接交互，全局信息捕捉效率高于逐层扩大感受野的 U-Net。

- **条件注入 (adaLN-Zero)**：时间步 t 与类别等条件信息通过自适应层归一化 (adaptive layer norm, adaLN-Zero) 注入每个 Transformer 块，引导去噪过程。
 - 值得注意的是，DiT 论文本身**不是文生图**，它做的是 ImageNet 上的**类别条件生成**，即：给一个类别编号（比如 281 号=橘猫），生成对应类别的图片。所以条件是 label 而不是自然语言。这篇论文的目的不是做一个产品级的文生图模型，而是**验证一件事**：把扩散模型的骨架从 U-Net 换成 Transformer 效果会怎样。



训练过程

DiT 的训练目标与 DDPM 一致，仍是噪声预测，差别只在于骨干网络与所处空间。

- **隐表示加噪**：取一张真图，先用冻结的 VAE 编码器压缩为隐表示 z_0 ，随机采样时间步 t ，按扩散调度向 z_0 加噪得到 z_t 。
- **patch 化与预测**：将 z_t 切成 patch token 序列，连同经 adaLN-Zero 注入的时间步与类别条件一起送入 Transformer，让其预测所加的噪声。
- **损失与更新**：预测噪声与真实噪声做 MSE，反向传播更新 Transformer 参数（VAE 保持冻结）。整个流程与 DDPM 的回归式训练完全同构，仅把 U-Net 替换为 Transformer。

生成过程

生成沿用扩散模型的标准迭代采样，在潜空间完成后再解码。

- **隐空间采样**：在潜空间随机采样一个纯噪声隐表示，切成 patch 序列，从 $t=T$ 开始让 Transformer 在条件引导下逐步预测并去除噪声，反复迭代至得到干净的隐表示 z_0 。
- **解码还原**：将 z_0 送入 VAE 解码器还原为最终图像。

核心贡献

- **在标准基准上超过了所有此前的扩散模型**。最大的 DiT-XL/2 模型在 256×256 ImageNet 上达到 FID 2.27, 大幅超越此前 LDM 创下的最好成绩 3.60。而且它还超过或追平了 BigGAN、StyleGAN-XL 等强力 GAN。

- 证明了 DiT 遵循与 LLM 类似的 **scaling law**——模型越大、计算越多，FID 越低。回答了一个关键问题：扩散模型的性能瓶颈不在 U-Net 架构本身，而在于可扩展性。

与 ViT 和 LDM 的关系

DiT = ViT 的 patch tokenization + LDM 的潜空间扩散 + Transformer 的 scaling 能力。它让扩散模型和自回归模型共享了 Transformer 这一基础架构，为两者的融合创造了条件。

地位与影响

DiT 是 Sora（OpenAI 视频生成模型）的架构基础，也是 FLUX、Seedream 等最新扩散模型的骨干。更重要的是，DiT 是扩散范式与自回归范式之间的桥梁。

四、自回归范式

自回归生成的核心思想是：把图像拆成一个序列，然后像语言模型生成文字一样，逐个预测下一个 token。这条路线 2016 年起步，中间沉寂了几年（被扩散模型压下去），2024 年起重新成为前沿焦点。以下 7 篇论文构成了完整的演进链。

什么是自回归（autoregressive）？

用“自己前面的部分”预测“自己的下一部分”：“自”指用自身已有的内容，“回归”指预测。最纯粹的例子是语言模型：写句子时，根据前面已写出的词预测下一个词。

"今天天气" → 预测 → "真"

"今天天气真" → 预测 → "好"

……一个词接一个词地往下写

其核心动作是：逐个生成，每生成一个新单元，都要回看已经生成的全部。自回归的灵魂在于：序列化、逐步推进、前面决定后面。

1. PixelRNN / PixelCNN (2016)

Pixel Recurrent Neural Networks — van den Oord et al. (DeepMind)

(2016) <https://arxiv.org/abs/1601.06759>



自回归生图的开山之作：像生成文字一样按顺序预测生成像素。

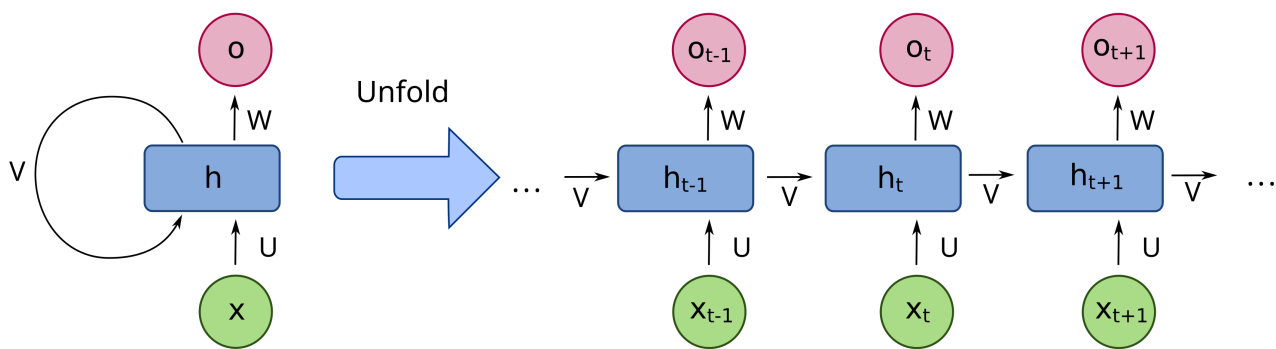
简介

这篇文章是自回归图像生成的开山之作。它把语言模型“逐词预测”的思路迁移到图像上：将图像按光栅扫描顺序（从左到右、从上到下）展开成一维像素序列，然后逐像素地预测——根据已经生成的所有像素，预测下一个像素的取值。整个范式的核心理念「**图像可以被当作序列来自回归生成**」即由此确立。

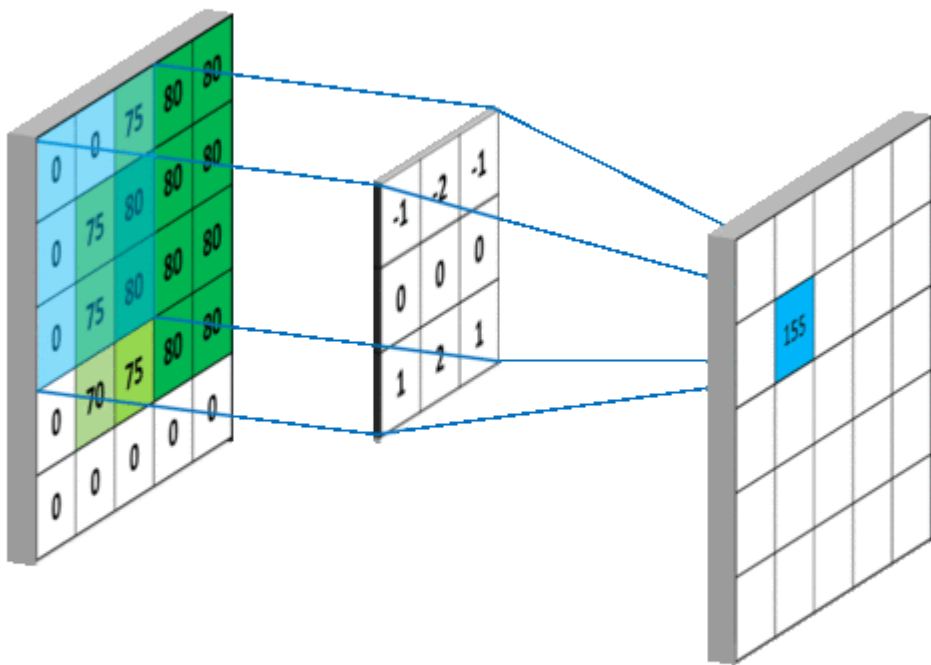
模型结构

论文提出了两类实现，区别在于"如何根据已生成像素预测下一个"：

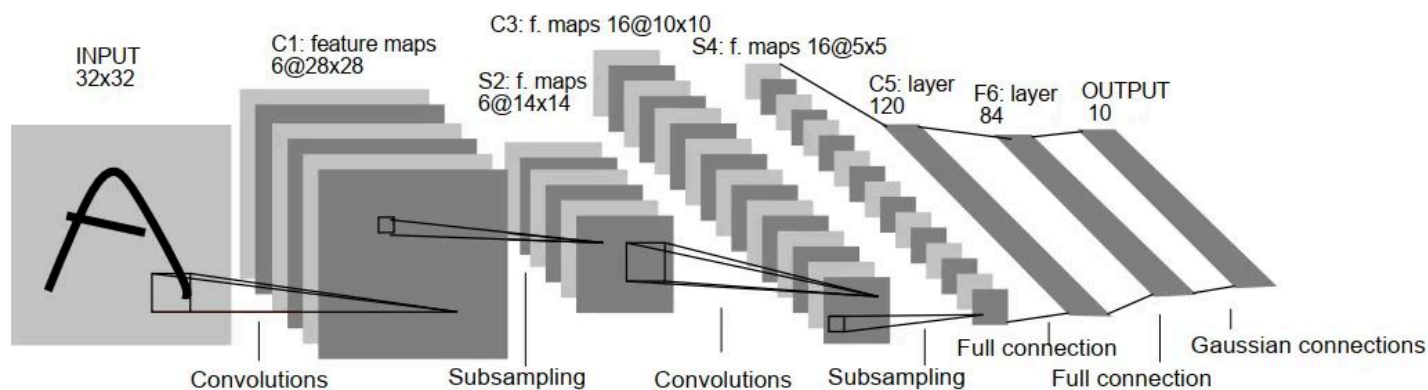
- **PixelRNN**：用循环神经网络（LSTM）建模像素间的长程依赖。沿序列逐步推进，理论上能记住此前所有像素的信息，建模能力强，但因循环结构难以并行，速度慢。
- **PixelCNN**：改用卷积来聚合上文信息，并引入掩码卷积（masked convolution）施加因果约束——预测某像素时，掩码会遮住该像素之后（尚未生成）的位置，确保网络只能"看到过去、看不到未来"，避免信息泄露。相比 PixelRNN 速度更快，但建模能力略逊。
- 输出形式：两者最终对每个像素输出一个在像素取值上的概率分布（将像素值当作离散类别预测），从该分布中采样得到具体像素值。



RNN（循环神经网络）



卷积操作



卷积神经网络结构（来自经典的 CNN 开山鼻祖 LeNet-5（1998））

训练过程

得益于因果约束的设计，**PixelCNN** 训练阶段**无需真正逐像素串行**（PixelRNN 还是得串行）。

PixelCNN 的训练：

- 监督信号：训练用的真图本身就提供了每个像素的"标准答案"，任务是让网络在"只看前面像素"的条件下，预测出当前像素的正确取值。
- 并行训练：由于真图的全部像素已知，掩码结构保证了每个位置只依赖其前序像素，因此整张图所有位置的预测可以一次性并行计算，再统一与真值比对、反向传播。损失为对每个像素分类的负对数似然。
- 关键点：训练能并行，是因为"前面的像素"在训练时已由真图给定；这一点与生成阶段必须串行形成鲜明对比。

生成过程

生成阶段无法并行，必须严格逐像素串行。

- 逐像素采样：从图像左上角第一个像素开始，网络依据已生成的像素预测当前像素的概率分布并采样，得到一个具体像素值；将其填入图中，作为下一步的"上文"，再预测下一个像素。
- 串行推进：按光栅扫描顺序一个接一个地生成，前一个像素未确定，后一个就无法开始，直到填满整张图。这正是该范式生成极慢的根源。

核心贡献

确立了「**图像可以被当作序列来自回归生成**」这一核心思想。PixelRNN 用 LSTM 建模像素间的依赖关系，PixelCNN 用带掩码的卷积实现因果约束，速度更快。

局限

逐像素生成极其缓慢，且难以捕捉全局结构。正因如此，这条路线在之后几年被 GAN 和后来的扩散模型盖过了风头。

地位与影响

整个自回归图像生成范式的思想源头。其"序列化生成"的理念被后续工作持续继承——VQ-VAE 将逐像素升级为逐 token，DALL·E 1 将其扩展到大规模文生图，VAR、LlamaGen 则在 2024 年让这条路线重回前沿。

参考文档

卷积神经网络：

<https://medium.com/thedeephub/convolutional-neural-networks-a-comprehensive-guide-5cc0b5eae175>

<https://mlnotebook.github.io/post/CNN1/>

<https://www.ibm.com/cn-zh/think/topics/convolutional-neural-networks?regionCode=cn&languageCode=zh&cm-history=cn-zh>

<https://cs231n.github.io/convolutional-networks/>

循环神经网络：

<https://cs231n.github.io/rnn/>

<https://www.geeksforgeeks.org/machine-learning/introduction-to-recurrent-neural-network/>

2. VQ-VAE (2017)

Neural Discrete Representation Learning — van den Oord et al. (DeepMind)
(2017) <https://arxiv.org/abs/1711.00937>



借用 VAE 思想，把「逐像素预测」升级成「逐 token 预测」。

简介

VQ-VAE (Vector Quantized Variational Autoencoder, 向量量化变分自编码器) 的关键创新，是用一个**离散码本** (codebook) 替代了 VAE 的连续潜空间：编码器输出的每个特征向量，都被替换为码本中与之最接近的向量，从而把一张图像压缩成**一串离散编号 (token)**。随后在这些离散 token 上训练一个自回归先验模型 (原文用 PixelCNN) 进行采样生成。它把自回归图像生成从「逐像素预测」解放到了「**逐 token 预测**」。

模型结构

VQ-VAE 的核心组成：

- **编码器 (CNN)**：将输入图像下采样为一个二维的特征向量网格 (如 32×32 个向量)，每个向量代表图像中一小块区域的特征。
- **码本**：维护一本固定大小的码本 (如含 512 个向量，各带一个编号)。编码器输出的每个向量，都被替换为码本中与其欧氏距离最近的那个向量——这一步称为**向量量化**。替换后，整张图被表示为一串码本编号 (离散 token)，连续潜空间由此离散化。
- **解码器 (CNN)**：接收量化后的向量网格，将其上采样、重建回原始分辨率的图像。

- 自回归先验模型（PixelCNN）**：在离散 token 序列上学习"下一个 token"的分布，用于生成阶段采样。

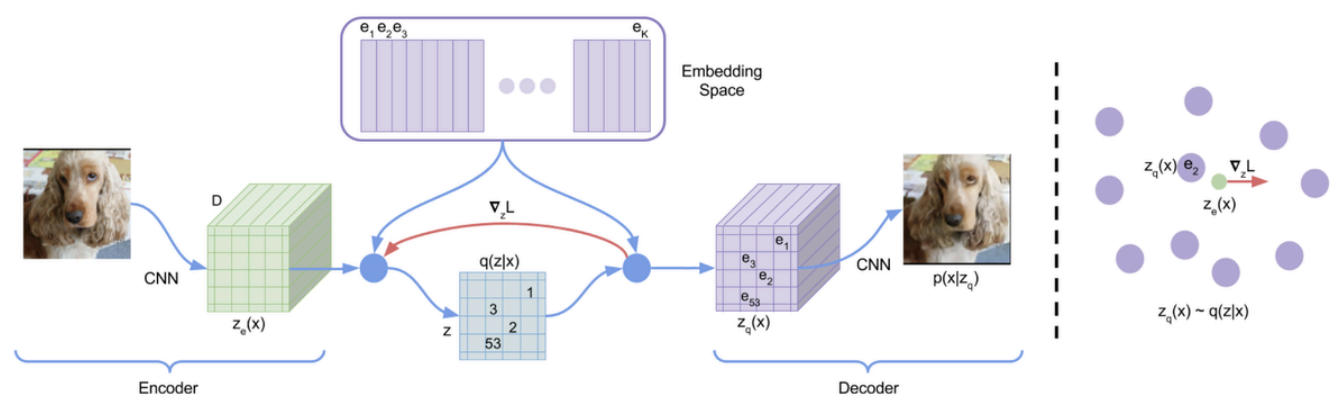


Figure 1: Left: A figure describing the VQ-VAE. Right: Visualisation of the embedding space. The output of the encoder $z(x)$ is mapped to the nearest point e_2 . The gradient $\nabla_z L$ (in red) will push the encoder to change its output, which could alter the configuration in the next forward pass.

VQ-VAE：图像 → 编码器 → 连续特征 → 向量量化（查表） → 离散特征 → 解码器 → 重建图像

涉及模型	是什么	输入 → 输出	何时训练	生成时用吗
编码器	CNN	图像 → 连续特征 $z_e(x)$	第一阶段	不用
解码器	CNN	量化特征 $z_q \rightarrow$ 重建图像	第一阶段	用
先验	PixelCNN	前面的 token → 下一个 token	第二阶段	用

训练过程

VQ-VAE 的训练分为先后两个阶段：先训好"图像 ↔ token"的转换器，再在 token 上训自回归先验。

- 第一阶段——训练编码器、解码器和码本**：让图像经"编码 → 量化 → 解码"后尽可能还原自身，以重建损失为主目标，同时更新编码器、解码器和码本。这里有一处技术难点：量化（"找最近的码本向量"）这一步不可导，梯度无法直接穿过。VQ-VAE 用直通估计（straight-through estimator）解决——前向传播照常做量化替换，反向传播时把解码器的梯度直接"拷贝"回编码器，绕过不可导的量化步骤；另用专门的损失项来更新码本向量和约束编码器输出。训练完成后，便得到一个能把任意图像编码为 token 串、又能从 token 串还原图像的"图像分词器"（tokenizer）。
- 第二阶段——训练自回归先验**：冻结上一阶段的编码器与码本，把训练集中的每张图都转成 token 序列，再在这些序列上训练一个 **PixelCNN**，学习「根据前面的 token 预测下一个 token」。此阶段与训练语言模型如出一辙，只是词表换成了视觉码本。

生成过程

生成沿"自回归生成 token 序列 → 解码还原图像"两步展开。

- **逐 token 自回归采样**：用训练好的 PixelCNN 先验，从头逐个生成 token——根据已生成的 token 预测下一个 token 的分布并采样，直到凑齐一整张图所需的 token 网格（如 32×32 ）。相比逐像素，token 数量少了一两个数量级，生成步数大幅减少。
- **解码还原**：将生成好的 token 序列按编号查回对应的码本向量，送入解码器，一步还原成完整图像。

核心贡献

把自回归模型从「逐像素预测」解放到了「逐 token 预测」。这是 VAE 和自回归模型的桥梁——借用 VAE 的编码解码框架，但用离散量化替代连续采样。

局限

- 单层量化，全局与局部难兼顾：原始 VQ-VAE 只有单一尺度的码本，难以同时刻画图像的全局结构与精细细节，生成质量与同期 GAN 仍有差距（这一点由后续的 VQ-VAE-2 通过多尺度层级量化改进）。
- 码本相关问题：训练中可能出现码本利用率低（大量码字几乎不被使用）等问题，影响表征效率。

地位与影响

后续几乎所有自回归图像生成工作的 **tokenizer** 都源自 VQ-VAE 的思想。没有它就没有后面的 VQGAN、DALL·E 1、LlamaGen。其**"离散视觉词典"**的理念，也是日后将图像纳入统一多模态序列、与文本 token 一同建模的关键前提。

3. VQ-VAE-2 (2019)

Generating Diverse High-Fidelity Images with VQ-VAE-2 — Razavi et al. (DeepMind)
(2019)<https://arxiv.org/abs/1906.00446>



在 VQ-VAE 基础上引入分层设计，大幅提升生成质量。

简介

VQ-VAE-2 在 VQ-VAE 的基础上引入多尺度层级结构来提升生成质量。其核心判断是：单层量化难以同时兼顾图像的全局结构与局部细节——颗粒太粗则丢失细节，太细则 token 过多。VQ-VAE-2 改用**两层量化**（顶层捕捉全局结构，底层捕捉局部细节），并为**每一层分别训练一个自回归先验**，从而在保持 token 数量可控的同时大幅提升保真度。

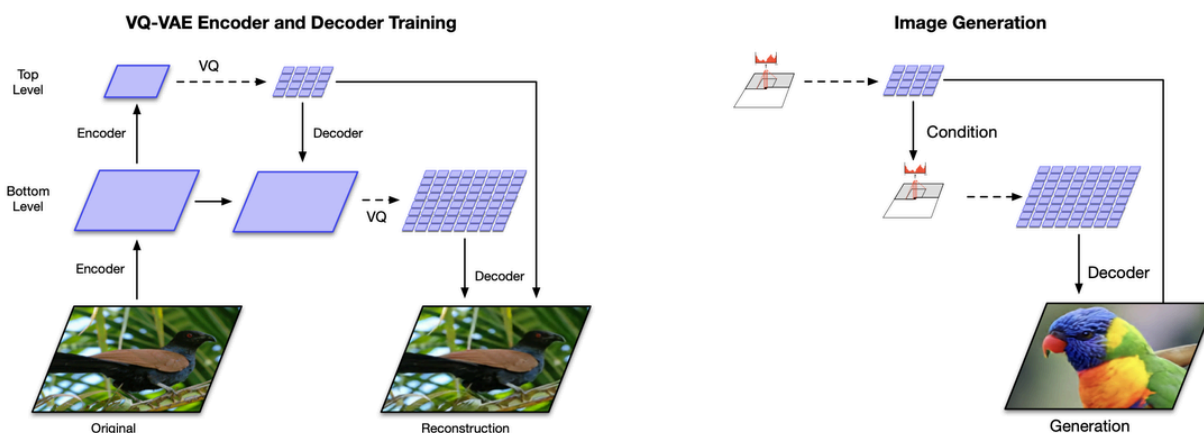
模型结构

VQ-VAE-2 把 VQ-VAE 的"单层量化"扩展为"分层量化"，整体仍是"编码 → 量化 → 解码"的框架，但量化分为两个尺度：

- **顶层（全局）**：颗粒较粗、token 数量较少，负责编码图像的整体结构与全局语义（如人脸朝向、物体大致形状与布局）。

- **底层（局部）**：颗粒较细、token 数量较多，负责编码精细的局部信息（如纹理、发丝、边缘）。底层在量化时以顶层信息为条件，使细节生成与全局结构保持一致。
- **解码器**：综合顶层与底层两套离散表示，共同重建出高分辨率图像。

此外，每一层各自配一个独立的自回归先验模型，用于生成阶段分层采样。



(a) Overview of the architecture of our hierarchical VQ-VAE. The encoders and decoders consist of deep neural networks. The input to the model is a 256×256 image that is compressed to quantized latent maps of size 64×64 and 32×32 for the *bottom* and *top* levels, respectively. The decoder reconstructs the image from the two latent maps.

(b) Multi-stage image generation. The top-level PixelCNN prior is conditioned on the class label, the bottom level PixelCNN is conditioned on the class label as well as the first level code. Thanks to the feed-forward decoder, the mapping between latents to pixels is fast. (The example image with a parrot is generated with this model).

Figure 2: VQ-VAE architecture.

训练过程

与 VQ-VAE 一样，VQ-VAE-2 也分两个阶段：先训分层自编码器，再训各层的自回归先验。

- **第一阶段——训练分层自编码器与码本**：让图像经"编码 → 顶层/底层两级量化 → 解码"后重建自身，以重建损失训练编码器、解码器与两层码本。量化不可导的问题仍沿用直通估计处理。训练完成后，得到一个能把图像编码为"顶层 + 底层"两套 token、并能据此还原图像的分层 tokenizer。
- **第二阶段——分层训练自回归先验**：冻结自编码器，把训练图像转成两层 token 序列，分别训练**两个自回归先验**：一个先验学习顶层 token 的分布（无条件，纯粹生成全局骨架）；另一个先验学习底层 token 的分布，且以顶层 token 为条件（在已知全局结构的前提下补充细节）。

生成过程

生成沿"先生成全局、再补充细节、最后解码"的层级顺序展开。

- **生成顶层 token**：先用顶层先验自回归地生成整张图的全局 token 序列，确定图像的整体结构。
- **条件生成底层 token**：再用底层先验，以刚生成的顶层 token 为条件，自回归地生成底层细节 token——细节的填充始终服从已确定的全局骨架，保证两者协调。
- **解码还原**：将顶层与底层 token 一并查回码本向量、送入解码器，重建出最终的高分辨率图像。

Algorithm 1 VQ-VAE training (stage 1)

Require: Functions E_{top} , E_{bottom} , D , $\mathbf{x}$
(batch of training images)

- 1: $\mathbf{h}_{top} \leftarrow E_{top}(\mathbf{x})$
 ▷ quantize with top codebook eq 1
 - 2: $\mathbf{e}_{top} \leftarrow \text{Quantize}(\mathbf{h}_{top})$
 - 3: $\mathbf{h}_{bottom} \leftarrow E_{bottom}(\mathbf{x}, \mathbf{e}_{top})$
 ▷ quantize with bottom codebook eq 1
 - 4: $\mathbf{e}_{bottom} \leftarrow \text{Quantize}(\mathbf{h}_{bottom})$
 - 5: $\hat{\mathbf{x}} \leftarrow D(\mathbf{e}_{top}, \mathbf{e}_{bottom})$
 ▷ Loss according to eq 2
 - 6: $\theta \leftarrow \text{Update}(\mathcal{L}(\mathbf{x}, \hat{\mathbf{x}}))$
-

Algorithm 2 Prior training (stage 2)

- 1: $\mathbf{T}_{top}, \mathbf{T}_{bottom} \leftarrow \emptyset$ ▷ training set
 - 2: **for** $\mathbf{x} \in \text{training set}$ **do**
 - 3: $\mathbf{e}_{top} \leftarrow \text{Quantize}(E_{top}(\mathbf{x}))$
 - 4: $\mathbf{e}_{bottom} \leftarrow \text{Quantize}(E_{bottom}(\mathbf{x}, \mathbf{e}_{top}))$
 - 5: $\mathbf{T}_{top} \leftarrow \mathbf{T}_{top} \cup \mathbf{e}_{top}$
 - 6: $\mathbf{T}_{bottom} \leftarrow \mathbf{T}_{bottom} \cup \mathbf{e}_{bottom}$
 - 7: **end for**
 - 8: $p_{top} = \text{TrainPixelCNN}(\mathbf{T}_{top})$
 - 9: $p_{bottom} = \text{TrainCondPixelCNN}(\mathbf{T}_{bottom}, \mathbf{T}_{top})$
 ▷ Sampling procedure
 - 10: **while** true **do**
 - 11: $\mathbf{e}_{top} \sim p_{top}$
 - 12: $\mathbf{e}_{bottom} \sim p_{bottom}(\mathbf{e}_{top})$
 - 13: $\mathbf{x} \leftarrow D(\mathbf{e}_{top}, \mathbf{e}_{bottom})$
 - 14: **end while**
-

核心贡献

VQ-VAE 只有单层量化，难以兼顾全局和局部。VQ-VAE-2 的层级设计让生成质量大幅提升，首次在 FID 等指标上接近了同期的 GAN。

局限

- 仍依赖独立的先验模型：生成质量受限于自回归先验（如 **PixelCNN**）的建模能力，且分层结构使训练与采样流程更复杂。
- 对抗/感知质量仍有差距：重建主要由像素级重建损失驱动，在感知真实感上仍不及引入对抗训练的方案（这一点由后续的 VQGAN 改进）。

地位与影响

证明了自回归路线在质量上有追赶 GAN 的潜力，也为后来 VQGAN 的进一步改进奠定了基础。

4. VQGAN (2021)

Taming Transformers for High-Resolution Image Synthesis — Esser et al.

(2021) <https://arxiv.org/abs/2012.09841>



在 VQ-VAE 的基础上引入 GAN 对抗训练和 Transformer 序列建模。

简介

VQGAN 把 **GAN 的对抗训练**引入 **VQ-VAE 的 tokenizer 训练**中。在 VQ-VAE 原有的重建损失之外，额外加上一个 PatchGAN 判别器损失和感知损失，让码本学到更丰富、更感知一致的视觉表征，从而显著提升图像离散化（压缩-还原）的质量。在生成端，则用 GPT-2 风格的 **Transformer 替代**

PixelCNN，在离散 token 序列上做自回归生成。可以概括为：**VQGAN = VQ-VAE 的离散量化 + GAN 的对抗训练 + Transformer 的序列建模**。

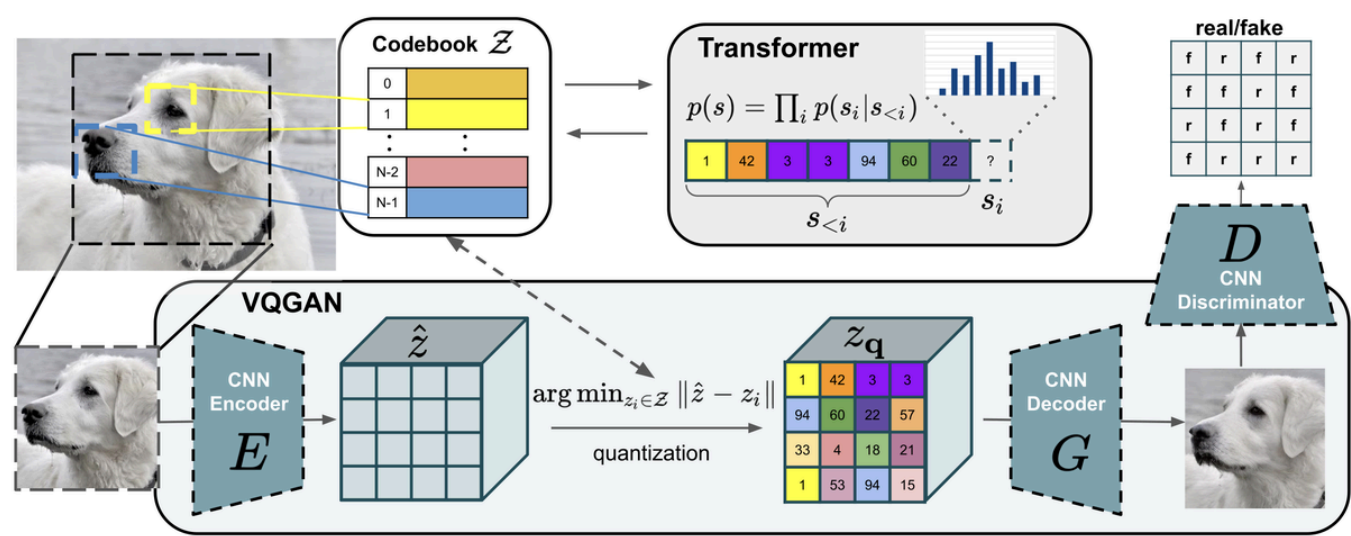


Figure 2. Our approach uses a convolutional VQGAN to learn a codebook of context-rich visual parts, whose composition is subsequently modeled with an autoregressive transformer architecture. A discrete codebook provides the interface between these architectures and a patch-based discriminator enables strong compression while retaining high perceptual quality. This method introduces the efficiency of convolutional approaches to transformer based high resolution image synthesis.

模型结构

- VQGAN 沿用"编码 → 量化 → 解码"的 VQ-VAE 框架，但在训练目标和先验模型两处做了关键升级：
- 编码器 / 码本 / 解码器：与 VQ-VAE 一致，负责把图像压缩为离散 token、再从 token 还原图像。
 - 判别器（来自 GAN）：额外引入一个 PatchGAN 判别器，对重建图像的局部块判断真假，专门用来提升还原图的感知真实感。它只在训练 tokenizer 时使用，生成阶段不参与。
 - 自回归先验（Transformer）：用 GPT-2 风格的 **Transformer** 替代 VQ-VAE 的 PixelCNN，在 token 序列上学习"下一个 token"的分布。相比 PixelCNN，Transformer 建模能力更强、扩展性（scaling）更好。

训练过程

- 与 VQ-VAE 一样分两个阶段，差别集中在第一阶段的训练目标，以及第二阶段的先验换成了 Transformer。
- **第一阶段——训练 tokenizer（含对抗训练）**：让图像经"编码 → 量化 → 解码"重建自身，但损失不再只是逐像素重建。VQGAN 同时施加三类损失：① 重建损失（保证还原大体正确）；② 感知损失（在特征层面比对，让结果更符合人眼感知）；③ 对抗损失（让 PatchGAN 判别器分清重建图与真图，逼出清晰锐利的细节）。三者共同训练编码器、解码器与码本，量化不可导的问题仍沿用直通估计处理。这一阶段产出一个质量远高于 VQ-VAE 的图像 tokenizer。
 - **第二阶段——训练 Transformer 先验**：冻结 tokenizer，把训练图像转成 token 序列，训练一个 GPT-2 风格的 Transformer 做标准的"下一个 token 预测"。此阶段与训练语言模型几乎完全相同，只是词表换成了视觉码本。

生成过程

生成沿"自回归生成 token 序列 → 解码还原图像"两步展开，与 VQ-VAE 同构，仅先验模型不同。

- 逐 token 自回归采样：用训练好的 Transformer 先验，从头逐个生成 token——依据已生成的 token 预测下一个 token 的分布并采样，直到凑齐一整张图所需的 token 网格。
- 解码还原：将生成的 token 序列查回码本向量，送入解码器，还原成最终图像。**得益于第一阶段更高质量的 tokenizer，最终图像在清晰度和细节上明显优于 VQ-VAE。**

核心贡献

VQGAN = VQ-VAE 的离散量化 + GAN 的对抗训练 + Transformer 的序列建模。相比 VQ-VAE-2 用 PixelCNN 做先验，VQGAN 用 Transformer 做先验的选择更具可扩展性。

局限

- 两阶段训练复杂：需先训含对抗损失的 tokenizer、再训 Transformer 先验，流程较长，且对抗训练本身带来 GAN 固有的不稳定性。
- 自回归生成仍偏慢：逐 token 串行采样的固有限制依旧存在，长 token 序列的生成速度受限。

地位与影响

VQGAN 成为自回归图像生成的标准 tokenizer 基线，至今很多工作仍在其基础上改进。它是三种范式交叉的经典产物——同时融合了 VAE 的离散量化框架、GAN 的对抗训练与 Transformer 的序列建模。

5. DALL·E 1 (2021)

Zero-Shot Text-to-Image Generation — Ramesh et al. (OpenAI)
(2021)<https://arxiv.org/abs/2102.12092>



第一个大规模的文生图自回归模型。

简介

DALL·E 1 是 **OpenAI** 在 2021 年 1 月发布的文本到图像生成模型，是把"**文本描述变成图像**"这件事第一次大规模、惊艳地展示给公众的工作之一。它把文本与图像统一成一条 token 序列，用单一的大型 Transformer 自回归地预测整个序列。架构由两部分组成：一个 **dVAE**（离散 VAE，作用类似 VQ-VAE，负责把图像压缩成离散 token）和一个 **120 亿参数的自回归 Transformer**。训练数据为 2.5 亿个文本-图像对。它第一次向公众直观展示了"用一句话生成任意图像"的能力。

DALL·E 1 和 VQ-VAE-2 对照看，关键变化就是：**把第二阶段的先验从 PixelCNN 换成了大规模 Transformer，并且把文本作为条件拼进了序列。**

模型结构

DALL·E 1 把"文生图"彻底简化为"序列建模"，核心由两个组件构成：

- **dVAE (图像 tokenizer)**：沿袭 VQ-VAE 的离散化思想，将图像编码为一串离散图像 token，并能从 token 还原回图像。它扮演的是"图像分词器"角色，把连续的图像翻译成 Transformer 能处理的离散符号。
- **自回归 Transformer (120 亿参数)**：系统的主体。它处理的是"**文本 token + 图像 token**"拼接而成的统一序列，任务是标准的"下一个 token 预测"，与 GPT 类语言模型在机制上完全一致。

训练过程

DALL·E 1 的训练分为两步：先得到图像 tokenizer，再在拼接序列上训练自回归 Transformer。

- **第一阶段——训练 dVAE**：在图像数据上训练 dVAE，使其能把图像压缩为离散 token 序列、并高质量还原。完成后 dVAE 固定，作为后续的图像分词器。
- **第二阶段——训练自回归 Transformer**：对每个图文对，先用文本分词器把文字编码为文本 token，再用 dVAE 把图像编码为图像 token，**将两者按"文本在前、图像在后"拼成一条长序列**。Transformer 在这条序列上做标准的"下一个 token 预测"——根据前面所有 token（含全部文本 token 与已出现的图像 token）预测下一个 token，以负对数似然反向传播。如此，模型学会了"读完一段文字描述后，接下来应当生成怎样的图像 token"。

生成过程

生成沿"文本 token 引导 → 自回归补全图像 token → 解码还原"展开。

- **文本编码**：将用户输入的 prompt 用文本分词器编码为文本 token，作为序列的开头（条件）。
- **自回归生成图像 token**：Transformer 以这些文本 token 为前缀，逐个预测后续的图像 token——每生成一个就并入序列、作为下文继续预测，直到凑齐一整张图所需的图像 token。
- **解码还原**：把生成的图像 token 交给 dVAE 解码器，还原成最终图像。整个流程与语言模型续写文本同构，差别仅在于"被续写"的是图像 token。

核心贡献

第一次向公众展示了「**用文字描述生成任意图像**」的能力，引爆了整个 text-to-image 赛道。

局限

- 生成质量不及同期扩散模型：自回归逐 token 生成的画质与一致性，很快被随后的扩散模型（如 GLIDE、DALL·E 2）反超。
- 推理较慢：逐 token 串行生成长图像序列，速度受限。
- 闭源：未开放模型权重，限制了学术复现。

6. VAR (2024)

Visual Autoregressive Modeling: Scalable Image Generation via Next-Scale Prediction — Tian et al. (北大 & 字节) (2024) <https://arxiv.org/abs/2404.02905>



重新定义了自回归的顺序，将「逐 token 预测」改为「逐尺度预测」；第一次全面超越 DiT。

简介

VAR 提出了一种全新的自回归"顺序"定义：不再按光栅扫描逐 token 预测，而是按分辨率尺度从粗到细预测（**next-scale prediction**，**下一尺度预测**）。先生成最低分辨率的 token map，再以其为基础逐级生成更高分辨率的 token map，直至最高分辨率。这一改动既保留了图像的二维空间结构，又大幅减少了生成步数，首次在 ImageNet 基准上全面超越 DiT。

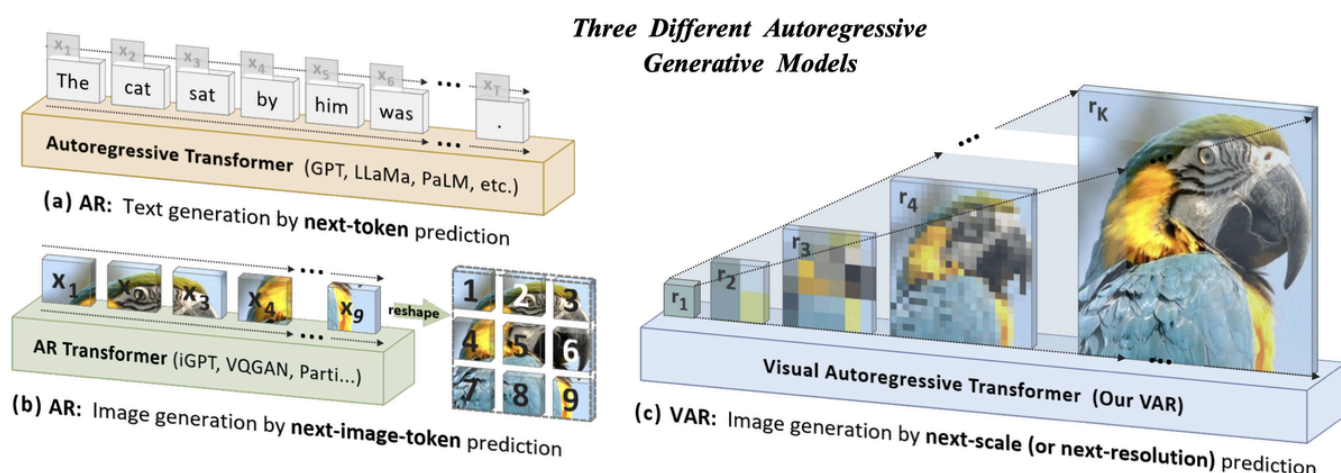


Figure 2: **Standard autoregressive modeling (AR) vs. our proposed visual autoregressive modeling (VAR).** (a) AR applied to language: sequential text token generation from left to right, word by word; (b) AR applied to images: sequential visual token generation in a raster-scan order, from left to right, top to bottom; (c) VAR for images: multi-scale token maps are autoregressively generated from coarse to fine scales (lower to higher resolutions), with parallel token generation within each scale. VAR requires a multi-scale VQVAE to work.

模型结构

VAR 仍是"tokenizer + 自回归 Transformer"的整体框架，但 tokenizer 和预测目标都围绕"多尺度"重新设计：

- **多尺度 tokenizer**：不再把图像编码为单一分辨率的 token map，而是编码为一组从粗到细、分辨率递增的 token maps（如 1×1 、 2×2 、 $4 \times 4 \dots$ 直到最高分辨率）。粗尺度承载全局结构，细尺度承载局部细节。
- **自回归 Transformer（按尺度展开）**：主体仍是 Transformer，但它自回归的单位是"一个完整的尺度"而非"一个 token"。预测某一尺度时，模型以此前所有更粗的尺度为条件，一次性并行生成当前尺度的全部 token。

训练过程

VAR 的训练目标从"预测下一个 token"改为"预测下一个尺度"，但仍是自回归式的条件预测。

- **训练多尺度 VQ 自编码器（tokenizer）**：用多尺度 tokenizer 把每张训练图编码为一组由粗到细的 token maps，作为各尺度的监督目标。

- 训练 VAR Transformer，用于 next-scale prediction（逐尺度预测）：** 训练 Transformer 在"已知前面所有粗尺度"的条件下，预测下一个（更细）尺度的全部 token。由于同一尺度内的 token 可并行预测，训练效率较逐 token 方式更高；损失仍为对 token 的分类（负对数似然）。模型借此学会"如何从粗略轮廓逐级细化出完整图像"。

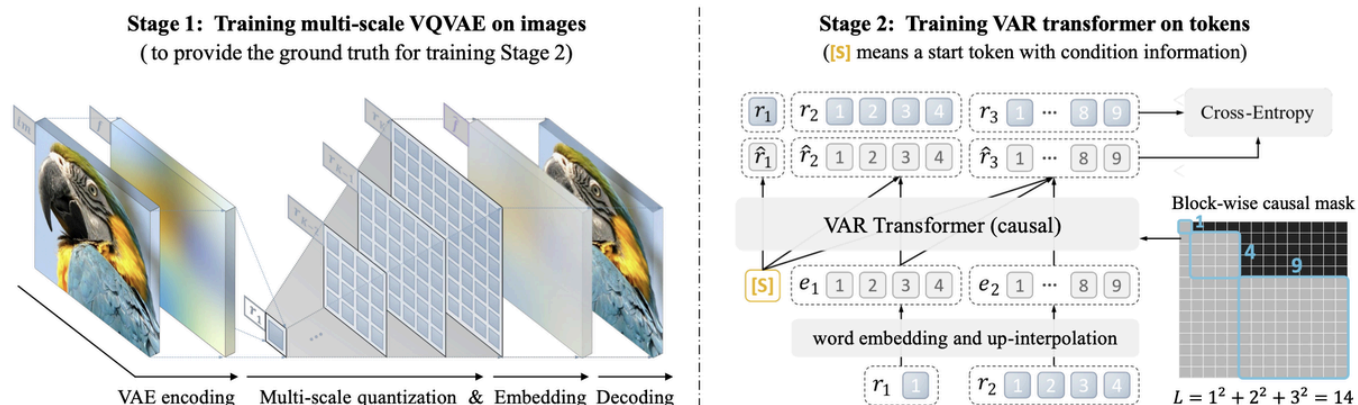


Figure 4: VAR involves two separated training stages. Stage 1: a multi-scale VQ autoencoder encodes an image into K token maps $R = (r_1, r_2, \dots, r_K)$ and is trained by a compound loss (5). For details on “Multi-scale quantization” and “Embedding”, check Algorithm 1 and 2. Stage 2: a VAR transformer is trained via next-scale prediction (6): it takes $([s], r_1, r_2, \dots, r_{K-1})$ as input to predict $(r_1, r_2, r_3, \dots, r_K)$. The attention mask is used in training to ensure each r_k can only attend to $r_{\leq k}$. Standard cross-entropy loss is used.

生成过程

生成沿"由粗到细、逐尺度生成"展开。

- 逐尺度生成：** 从最低分辨率尺度起步，自回归地生成整张图的全局 token map；再以已生成的所有粗尺度为条件，依次生成更高分辨率的 token map，一级一级补充细节，直到最高分辨率。**每个尺度内部的 token 一次性并行产出，因此总步数仅为尺度数量，远少于逐 token 方式。**
- 解码还原：** 将最终的多尺度 token 经 tokenizer 解码，还原为完整图像。

核心贡献

传统自回归把 2D 图像强行拉成 1D 序列，丢失了空间结构信息。VAR 保留了 2D 空间关系，用多尺度层级替代了一维序列。首次在 ImageNet 基准上全面超越了 DiT，并验证了与 LLM 类似的 scaling laws。

局限

- 依赖专门的多尺度 tokenizer：整套方法的效果与多尺度量化的设计强相关，tokenizer 的构建比单尺度方案更复杂。
- 主要在类别条件 / ImageNet 上验证：原文以类别条件生成为主，向大规模开放域文生图的扩展仍需后续工作检验。

地位与影响

NeurIPS 2024 最佳论文。它重新定义了「自回归」在视觉领域应该怎么做，可能是这条路线上最重要的范式革新。

7. LlamaGen (2024)

💡 证明了即使不做任何和 VAR 类似的优化改造，直接暴力使用最标准的 LLM 架构也能超过扩散模型。

Autoregressive Model Beats Diffusion: Llama for Scalable Image Generation — Sun et al.
(2024) <https://arxiv.org/abs/2406.06525>

简介

LlamaGen 用几乎与 Llama（Meta 的大语言模型）**完全相同的架构来做图像生成**，不添加任何视觉特有的归纳偏置。它沿用 DALL·E 1 式的"图像 token 化 + 标准下一个 token 预测"路线，模型从 1.11 亿个参数扩展到 **31 亿**个参数，在 ImageNet 256×256 上达到 FID 2.18，超越了 LDM 和 DiT。其核心论点是：纯粹的 LLM 架构无需任何修改，即可胜任图像生成。

模型结构

LlamaGen 刻意把结构压到最简，核心只有两件标准组件：

- **图像 tokenizer**：一个高质量的（VQGAN 式）离散 tokenizer，把图像压缩为离散 token 序列、并能还原。论文强调 tokenizer 的质量是成败关键之一。
- **Llama 架构的自回归 Transformer**：直接采用 Llama 的标准设计（如 RMSNorm、SwiGLU、RoPE 等），不为图像添加任何专门改动，任务是标准的"下一个 token 预测"。正因架构与主流 LLM 一致，它能直接复用 LLM 生态的训练与推理工具。

训练过程

训练与标准语言模型几乎无异，分 tokenizer 与自回归模型两步。

- **第一阶段——训练 tokenizer**：训练一个高质量的离散图像 tokenizer，使其能把图像编码为 token 序列并高保真还原。tokenizer 的重建质量直接决定了生成质量的上限，是本工作的重点之一。
- **第二阶段——训练 Llama 式 Transformer**：把训练图像（及类别/文本条件）转成 token 序列，让 Llama 架构的 Transformer 做标准的"下一个 token 预测"，以负对数似然反向传播。整个训练流程与训练一个语言模型完全同构，无需任何视觉特有的改动，因而可直接套用成熟的 LLM 训练框架。

生成过程

生成沿用语言模型的逐 token 自回归续写，与 DALL·E 1 同构。

- **逐 token 自回归采样**：以条件（类别或文本 token）为前缀，Transformer 逐个预测后续图像 token，每生成一个便并入序列继续预测，直到凑齐整张图所需的 token。
- **推理加速与解码**：由于架构与 LLM 一致，可直接借用 vLLM 等成熟推理优化工具，实现约 3–4 倍的推理加速；最后将生成的 token 经 tokenizer 解码还原为图像。

核心贡献

核心发现是：只要 tokenizer 够好、数据质量够高，纯粹的 LLM 架构不需要任何修改就能做好图像生成。直接复用了 LLM 的训练框架、推理优化（vLLM 等），实现了 3-4 倍的推理加速。

局限

- 质量受 tokenizer 上限制约：生成质量高度依赖离散 tokenizer 的重建能力，tokenizer 成为整个系统的瓶颈。
- 逐 token 生成仍偏慢：即便有推理加速，标准自回归的串行生成相比扩散的并行去噪在长序列上仍有速度劣势。

与 VAR 的关系

VAR 证明了「换一种自回归顺序」可以超越扩散模型；LlamaGen 证明了「用最标准的 LLM 架构 + 标准的 next-token prediction」也能超越扩散模型。两者从不同角度证实了自回归范式的潜力。

地位与影响

它是「自回归图像生成可以直接搭 LLM 便车」这个论点最有力的实证，为文本和图像的统一生成模型（如 Emu3、Chameleon）提供了架构层面的信心。它也呼应了全文的大趋势——当图像被当作一种“语言”来处理，所有模态便都在向 Transformer 这一统一架构收敛。

核心贡献

- 证明纯 LLM 架构无需修改即可超越扩散：不加任何视觉归纳偏置，仅靠标准 Llama 架构 + 高质量 tokenizer，在 ImageNet 256×256 上以 FID 2.18 超越 LDM 与 DiT。
- 直接复用 LLM 生态：训练框架与推理优化（如 vLLM）拿来即用，实现约 3-4 倍推理加速，工程成本大幅降低。

五、大厂如何押注技术路线

注意：这两家的最新模型都不公开核心架构、训练配方和参数量。因此下文对原理的描述部分基于官方有限披露、部分基于行业共识推断。

1. OpenAI



反复横跳。图像从自回归转向扩散再转向自回归；视频（Sora）纯扩散。

技术演进脉络：

- DALL·E 1(2021,[arXiv:2102.12092](#))——纯自回归。dVAE + 120 亿参数 Transformer,把图当 token 序列预测。
- DALL·E 2(2022,[arXiv:2204.06125](#))——掉头转扩散。unCLIP 架构(Prior + 扩散 Decoder)。当时扩散在质量上碾压自回归,OpenAI 务实地放弃了自家开创的 AR 路线。

- **DALL·E 3**(2023)——仍是**扩散**。核心创新不在架构而在数据:用约 95% 的合成详细字幕训练,大幅提升指令遵循。但它本质是独立模型,在 ChatGPT 里靠 pipeline 拼接(LLM 改写 prompt → 丢给 DALL·E 3 画),语言和画图是两套割裂系统。
- **GPT-4o 原生图像 / GPT Image**(2025 起)——又掉回**自回归**,且真正"原生嵌入"在全模态 LLM 里。OpenAI 在 [GPT-4o 系统卡补充文档](#)中**明确**写了它是自回归、而非扩散。发布会白板那句"用一个大的自回归 Transformer 直接建模 $p(\text{text}, \text{pixels}, \text{sound})$ "就是纲领。[VTBench](#) 从行为侧佐证它是 AR:它会把图里读不清的文字"脑补"成通顺的词——这种"语言先验幻觉"正是图文 token 共享序列、被语言模型先验左右的典型表现。**业界普遍推测**它实际是 AR 骨干 + 扩散解码器的混合(先定粗结构、再逐行细化),但 OpenAI 未确认。

模型演进脉络:

- **GPT-4o 原生图像** —— 2025 年 3 月 —— 在 ChatGPT 上线内置生图,引爆全民生图,首周即有逾 1.3 亿用户生成超 7 亿张图。
- **gpt-image-1** —— 2025 年 4 月 —— 以 API 形式开放,自回归(确认)。
- **gpt-image-1.5** —— 2025 年 12 月 —— 在 ChatGPT 中称 ChatGPT Images,强化指令遵循与文字渲染。
- **gpt-image-2 (ChatGPT Images 2.0)** —— 2026 年 4 月 21 日 —— 引入 thinking(推理)模式、2K 分辨率、8 张批量、可先联网搜资料再作图,登顶 Image Arena(领先第二名一度 +242 分,创该榜最大领先纪录)。**注意:** OpenAI 研究者对 1.5 / 2 拒绝说明是扩散还是自回归,只称其为"通才模型 / GPT for images"——所以严格说,只有初代 GPT-4o 原生图像的 AR 范式是官方确认的,后续两代属未确认。(WaveSpeed 技术拆解)

2. Google



两边押注。图像有**纯扩散 (Imagen)**和**自回归 (Nano Banana)**两条并行线;视频 (Veo) 是扩散。

两条图像线:

- **Imagen 3 / 4:**经典**纯扩散**,主打照片级真实感、人像、产品图。
- **Nano Banana 系列:****自回归主导**——用 Gemini LLM 当文本编码器,直接生成自回归图像 token(每张约 1290 个)喂给图像解码器,因此深度继承 LLM 的推理和世界知识(同样存在 AR+扩散混合的推测)。[Max Woolf 的拆解](#)指出,其强项的秘密正是这个"既理解 prompt 又生成图像 token"的文本编码器。

模型演进脉络:

- **Gemini 2.5 Flash Image(初代 Nano Banana)** —— 2025 年 8 月 —— 以匿名代号"nano-banana"空降 LMArena,盲测击败 Midjourney、Flux,是 Arena 上投票量最大、最经受检验的型号之一。
- **Gemini Diffusion** —— 2025 年 5 月(I/O,候补名单制) —— **扩散式语言模型**,不逐词生成,而是像图像扩散那样从噪声迭代refine整块文本,约 1479 tokens/秒,擅长代码/数学/编辑。([DeepMind 官方页](#))
- **Nano Banana Pro = Gemini 3 Pro Image** —— 2025 年 11 月发布、2026 年 5 月 28 日 API GA(6 月企业版 GA) —— 旗舰,主打 4K、复杂版面、世界知识、可靠多语种文字渲染,以 Gemini 3 Pro 为编码器、强制"thinking"。([Google 官方博客](#))
- **Nano Banana 2 = Gemini 3.1 Flash Image** —— 2026 年 5 月 28 日 GA —— 高吞吐、实时场景的轻量高速版,GA 同日新增视频转图能力。([Google Cloud 公告](#))
- **DiffusionGemma** —— 2026 年(开源权重) —— 把 Gemini Diffusion 的研究做成开放权重版,建在 Gemma 家族上(MoE,约 26B 总参 / 3.8B 激活),H100 上 >1000 tok/s,用速度换质量,主打代码补全/内联编辑。

值得注意的是：Google 单独做了扩散语言模型(Gemini Diffusion / DiffusionGemma)，是三家里唯一在语言上押注扩散的。

六、思考

1. **模型不仅会越来越强，迭代速度也会越来越快，Agent 化的趋势也会越来越明显。**
 - Agent 化趋势：gpt-image-2 支持先思考再行动、支持联网搜、生成完成后会自检纠错
 - 对开发者来说，很多原本属于工程的工作正在不可避免地贬值。
2. **没有悲观的必要**
 - 「最好的模型」不等于「最受欢迎的产品」，正如「最好的产品」「最佳的内容」往往不等于「最佳的销量 / 流量」。一方面，绝大多数普通用户并不追求顶尖资源，反而是「够用就行」，因此完全没有必要对模型的碾压感到悲观；另一方面，产品功能和使用体验仍起到非常关键的决定性作用（参考近半年大众从 claude code cli 到 codex 客户端的转向）
3. **拥抱现实**
 - a. **防守**：传统软件传统功能的高度确定性、速度、隐私、成本、使用惯性等优势。
 - b. **耕耘**：「强模型能力」和「用户对结果满意」之间的差距。
 - c. **进攻**：有了模型能力作为支持，可以探索许多原来无法实现的产品功能，或者在模型难以取代的存量功能上借助模型能力进行优化；另外，有了 AI 提效，我们得以有机会真正实现软件开发的快速迭代 / 验证。
4. **如何耕耘「强模型」和「用户满意」之间的差距**

- 通用模型给出的必然是「通用最优解」，而用户要的是「对我而言的对」

5. 师夷长技以制夷：要借助模型创造出什么样的能力才能卷过模型 / 竞品？

- a. 尽量落在模型难以彻底包圆的领域
- b. 和竞品存在差异化 / 明显优于竞品
- c. 并且要能真正解决用户实际存在的核心痛点

6. 关于存量功能与模型能力的结合

- a. 不要硬上大模型：
 - i. 产品决策上：是否真正带来增量；是否真正解决问题；是否符合用户习惯；是否适合做成对话式入口……
 - ii. 技术决策上：LLM / VLM 最大的工程弊端在于耗时和不确定性。
- b. 什么时候上大模型：
 - i. 只有大模型才能做的场景：开放语义、模糊意图理解、跨模态推理、生文生图等等。
 - ii. 耗时相对不敏感，起码接受秒级别延迟（对于耗时敏感场景，模型也可以作为决策节点介入）
 - iii. 上之前控制好耗时 / 稳定性问题（耗时优化：压缩输入图、减少输出长度、采用轻量模型等）
 - iv. 模型增益不一定要体现在 C 端产品上，也可以体现在产品背后

7. For Agent 开发：技术层面具体要做好什么

- 把效果做好：对齐业内最先进的 agent 工程化实现（学最好的，不要自己从 0 摸索试错）+ 技术精细化与技术创新（并非所有通用最佳实践都适合于具体的业务场景）
- 把链路做稳：管好 bug、稳定性、耗时（非常基础，不一定能决定成功但搞不好能决定失败）
- 支持快速迭代：积极探索 AI 提效、闭环验证；探索产研协同迭代模式；做好基建（模型两个月一发版、AI 跑 demo 如火如荼的时代下，更需要快速验证试错，而技术在这个环节起到更关键的作用）
- 提供技术视角输入 + 补齐自身视野和技能短板：技术相对不了解产品，产品相对不了解技术，但是 AI 时代的许多决策都很需要同时了解产品 + 技术，所以可以主动提供这一块输入，同时主动沉淀自身的产品思维；另外即使是在技术领域内部，未来想要闭环验证一件事情，也可能需要一些算法认知 + 跨端工程实践能力，所以可以主动补齐自己的视野 / 技能短板。